BUKU PANDUAN UTAMA: ARSITEKTUR ANONIMITAS & OPSEC

Panduan Lengkap Rekayasa Privasi Siber dari Layer Terendah ke Tertinggi, Analisis Ancaman, Troubleshooting Lapangan, dan Skrip Otomatisasi Mandiri

Penyusun : Mohammad Bagus Satrio
Topik : Anonimitas Berlapis, Operations Security (OPSEC) & Anti-Tracking
Platform Lab : Kali Linux 2026.2 (VirtualBox) - User: nanonim / root

A. Fondasi Konsep: Piramida Pertahanan Anonimitas Berlapis

Anonimitas sejati dalam dunia cybersecurity bukanlah sekadar menyalakan mode Incognito browser atau menggunakan VPN komersial biasa. Pelacakan jejak digital terjadi secara terstruktur pada setiap tingkatan model OSI dan arsitektur sistem operasi. Jika satu lapisan bocor, seluruh identitas operator di dunia nyata dapat dibongkar oleh analis forensik.

Buku panduan ini mengurutkan implementasi dari lapisan paling mendasar (Layer terendah) hingga ke lapisan interaksi aplikasi dan perilaku (Layer tertinggi):

- 1. Layer 2 (Data Link): Alamat fisik kartu jaringan (MAC Address) yang dicatat router/switch lokal.
- 2. Layer 3 (Network): Alamat IP publik yang diberikan penyedia internet (ISP) dan dicatat firewall target.
- 3. Layer 4 (Transport): Protokol koneksi soket (TCP/UDP) dan kebijakan port tunneling.
- 4. Layer 7 (Application/DNS): Resolusi nama domain yang rawan bocor secara telanjang ke ISP lokal.
- 5. Kernel Layer: Kebijakan filter paket terpusat (iptables) untuk mencegah kebocoran paket liar non-proxy.
- 6. Browser Fingerprinting Layer: Perlindungan sidik jari grafis (Canvas) dan penonaktifan protokol WebRTC.
- 7. File & Metadata Layer: Sterilisasi tag tersembunyi (EXIF, author, dan koordinat satelit GPS) pada dokumen.
- 8. ISP Obfuscation & Behavioral Layer: Penyamaran pola jabat tangan Tor dari inspeksi DPI dan pembersihan residu RAM.

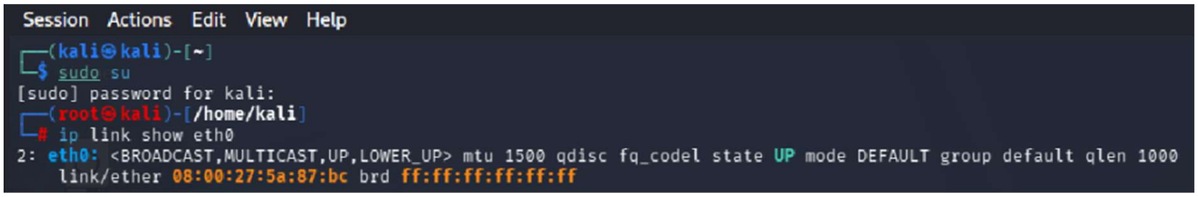
Modul 1: Layer 2 (Data Link) - Penyamaran Identitas Fisik (MAC Address Spoofing)

1.1. Mengapa MAC Address Berbahaya?

MAC Address adalah 12 karakter heksadesimal unik yang tertanam pada chip kartu jaringan fisik. Tiga blok awal (OUI - Organizationally Unique Identifier) mencerminkan nama pabrikan kartu jaringan. Jika Anda terhubung ke Wi-Fi publik tanpa menyamarkannya, router lokal menyimpan log permanen mengenai perangkat fisik Anda.

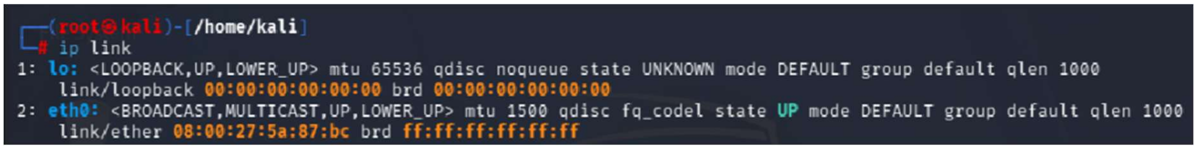
1.2. Langkah 1: Memeriksa MAC Address Asli:

```
$ ip link show eth0
```



```
┌──(kali㉿kali)-[~]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali]
└─# ip link show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
```

Screenshot 1: Pengecekan MAC address asli pada interface eth0



```
┌──(root㉿kali)-[/home/kali]
└─# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
```

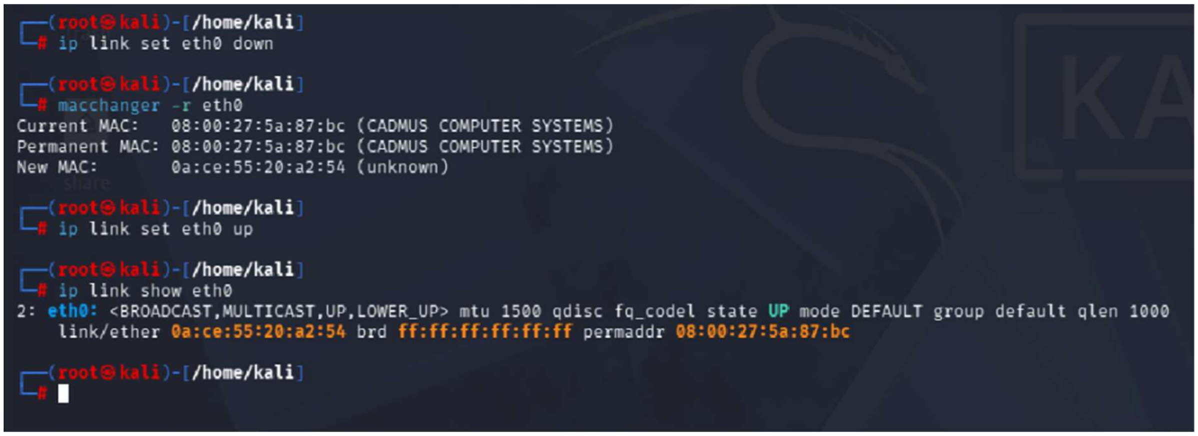
Screenshot 2: Daftar seluruh interface jaringan lokal (lo dan eth0)

- **Analisis Forensik:** 08:00:27:5a:87:bc. Tiga pasang heksa awal (08:00:27) merupakan OUI resmi milik Oracle VirtualBox. Ini adalah bukti telak bagi analis forensik bahwa sistem berjalan di atas mesin virtual.

1.3. Langkah 2: Mengacak MAC Address dengan `macchanger`:

Untuk mengubah MAC address, kartu jaringan harus dimatikan sementara, diacak dengan algoritma pseudorandom, lalu dihidupkan kembali:

```
$ ip link set eth0 down
macchanger -r eth0
ip link set eth0 up
ip link show eth0
```



```
┌──(root㉿kali)-[/home/kali]
└─# ip link set eth0 down

┌──(root㉿kali)-[/home/kali]
└─# macchanger -r eth0
Current MAC:   08:00:27:5a:87:bc (CADMUS COMPUTER SYSTEMS)
Permanent MAC: 08:00:27:5a:87:bc (CADMUS COMPUTER SYSTEMS)
New MAC:       0a:ce:55:20:a2:54 (unknown)

┌──(root㉿kali)-[/home/kali]
└─# ip link set eth0 up

┌──(root㉿kali)-[/home/kali]
└─# ip link show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 0a:ce:55:20:a2:54 brd ff:ff:ff:ff:ff:ff permaddr 08:00:27:5a:87:bc

┌──(root㉿kali)-[/home/kali]
└─# █
```

Screenshot 3: Hasil pengacakan MAC address dengan macchanger

- **Current MAC:** Alamat fisik bawaan sebelum diubah (08:00:27:5a:87:bc - CADMUS COMPUTER SYSTEMS).

- **Permanent MAC:** Alamat permanen perangkat yang tersimpan pada firmware ROM. Begitu VM di-restart, MAC asli ini pulih otomatis.

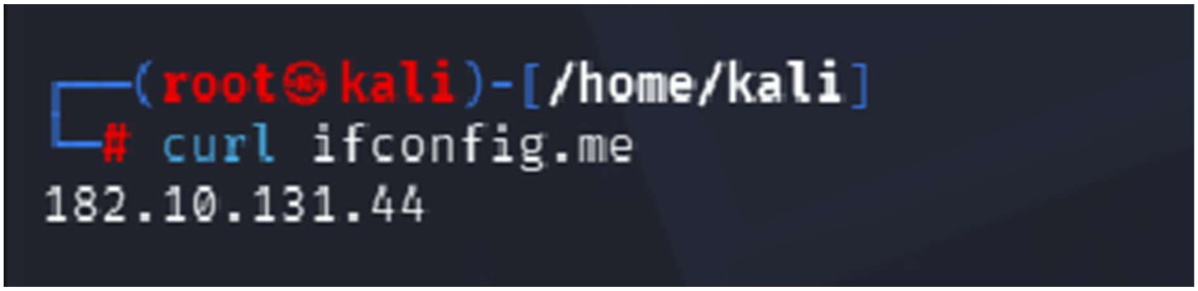
- **New MAC:** Identitas baru hasil pengacakan (0a:ce:55:20:a2:54). Router lokal kini hanya melihat alamat acak ini tanpa mengetahui perangkat fisik sebenarnya.

Modul 2: Layer 3 (Network) - Penyamaran Lalu Lintas Jaringan (Tor Multi-Hop Routing)

2.1. Memeriksa IP Asli ISP Rumah (Benchmark Awal):

Sebelum menyalakan proxy, catat IP publik asli yang diberikan oleh ISP penyedia koneksi:

```
$ curl ifconfig.me
```



```
┌──(root㉿kali)-[/home/kali]
└─# curl ifconfig.me
182.10.131.44
```

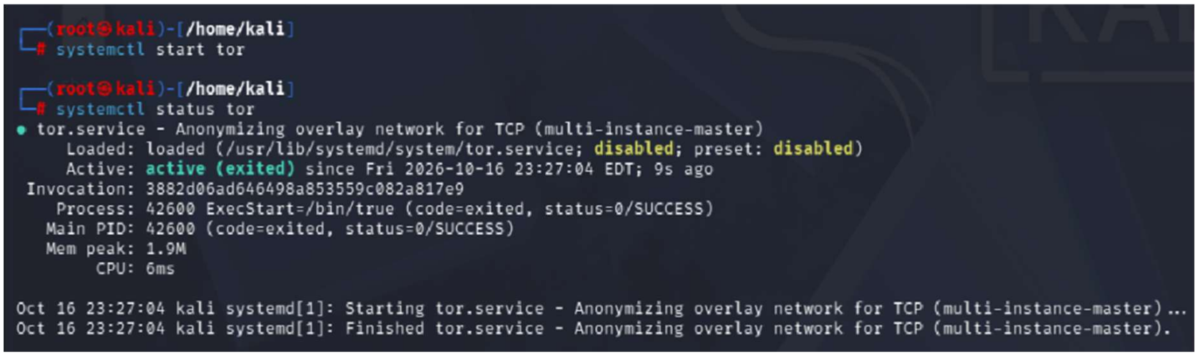
Screenshot 4: Pengecekan alamat IP publik asli (182.10.131.44)

- **Analisis:** IP publik asli rumah tercatat 182.10.131.44 (Indonesia). Jika melakukan pengujian tanpa lapisan perantara, target dapat langsung menelusuri lokasi geografis dan provider internet penyerang.

2.2. Mengaktifkan Layanan Tor Daemon:

Tor daemon membuka SOCKS5 proxy lokal pada 127.0.0.1:9050 dan membangun sirkuit relai 3 lapis terenkripsi (Entry/Guard -> Middle -> Exit Node):

```
$ systemctl start tor
systemctl status tor
```



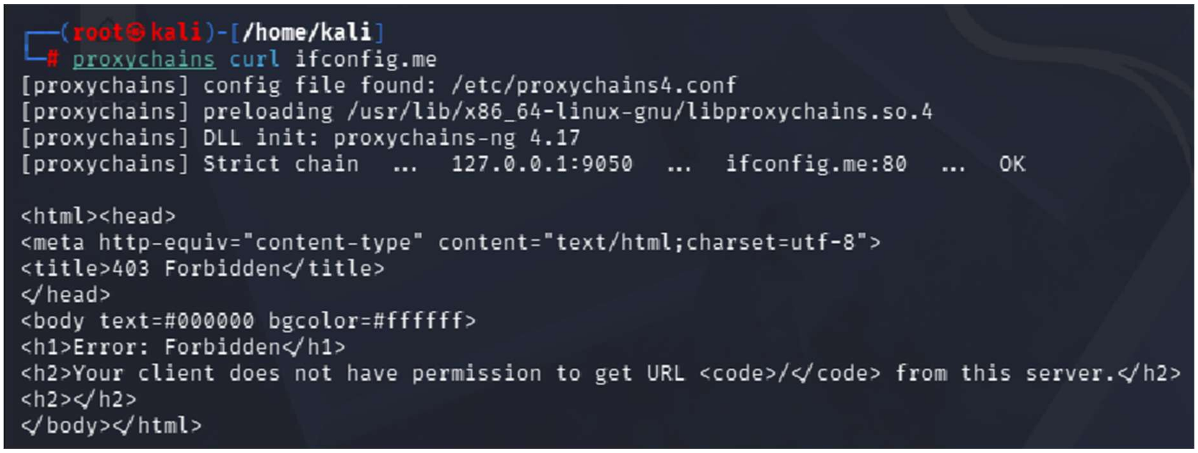
```
┌──(root㉿kali)-[/home/kali]
└─# systemctl start tor

┌──(root㉿kali)-[/home/kali]
└─# systemctl status tor
● tor.service - Anonymizing overlay network for TCP (multi-instance-master)
     Loaded: loaded (/usr/lib/systemd/system/tor.service; disabled; preset: disabled)
     Active: active (exited) since Fri 2026-10-16 23:27:04 EDT; 9s ago
 Invocation: 3882d06ad646498a853559c082a817e9
    Process: 42600 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 42600 (code=exited, status=0/SUCCESS)
   Mem peak: 1.9M
        CPU: 6ms

Oct 16 23:27:04 kali systemd[1]: Starting tor.service - Anonymizing overlay network for TCP (multi-instance-master)...
Oct 16 23:27:04 kali systemd[1]: Finished tor.service - Anonymizing overlay network for TCP (multi-instance-master).
```

Screenshot 5: Status master service Tor berjalan aktif (Active: active/running)

2.3. Mengarahkan Traffic dengan Proxychains:

Proxychains menggunakan teknik dynamic linking hook (LD_PRELOAD) untuk mencegat panggilan sistem jaringan (socket connect) dan membelokkannya ke Tor:

```
$ proxychains curl ifconfig.me
```



```
┌──(root㉿kali)-[/home/kali]
└─# proxychains curl ifconfig.me
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain  ...  127.0.0.1:9050  ...  ifconfig.me:80  ...  OK

<html><head>
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<title>403 Forbidden</title>
</head>
<body text=#000000 bgcolor=#ffffff>
<h1>Error: Forbidden</h1>
<h2>Your client does not have permission to get URL <code>/</code> from this server.</h2>
<h2></h2>
</body></html>
```

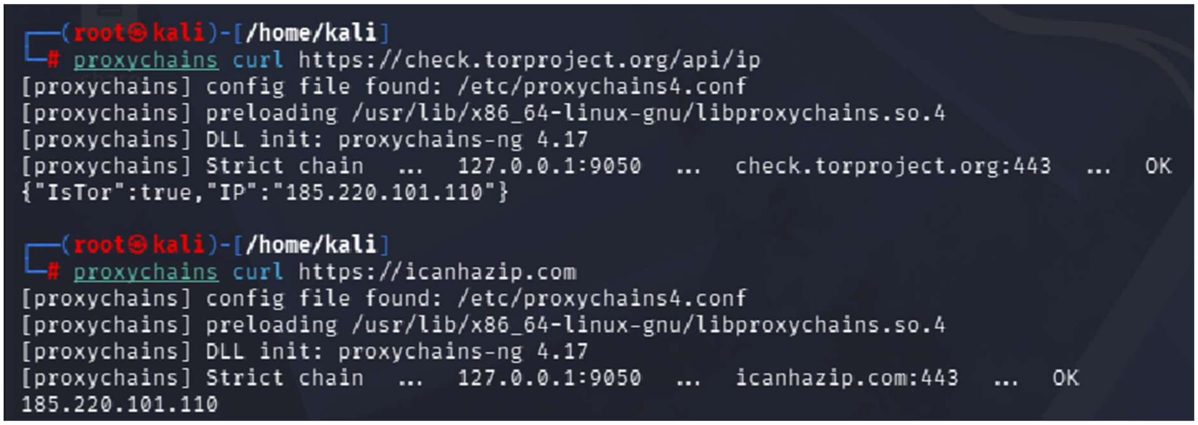
Screenshot 6: Proxychains merutekan paket via 127.0.0.1:9050

📌 **MEMAHAMI RESPON 403 FORBIDDEN:** Ketika server ifconfig.me merespon dengan '403 Forbidden', ini bukan kegagalan konfigurasi, melainkan bukti keberhasilan perutean. Server target mendeteksi bahwa IP penanya terdaftar dalam daftar hitam (blacklist) Tor Exit Node publik sehingga koneksi ditolak.

2.4. Verifikasi Status Tor Melalui Endpoint Resmi:

Gunakan endpoint resmi Tor Project untuk memvalidasi keberhasilan rute anonim:

```
$ proxychains curl https://check.torproject.org/api/ip
proxychains curl https://icanhazip.com
```



```
┌──(root㉿kali)-[/home/kali]
└─# proxychains curl https://check.torproject.org/api/ip
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain  ...  127.0.0.1:9050  ...  check.torproject.org:443  ...  OK
{"IsTor":true,"IP":"185.220.101.110"}

┌──(root㉿kali)-[/home/kali]
└─# proxychains curl https://icanhazip.com
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain  ...  127.0.0.1:9050  ...  icanhazip.com:443  ...  OK
185.220.101.110
```

Screenshot 7: Bukti status IsTor: true dan identitas IP baru (185.220.101.110)

- **Hasil:** IP publik berubah menjadi 185.220.101.110 (Exit Node berlokasi di Jerman) dengan respon status {"IsTor":true}. Identitas negara dan ISP asal tertutup sempurna.

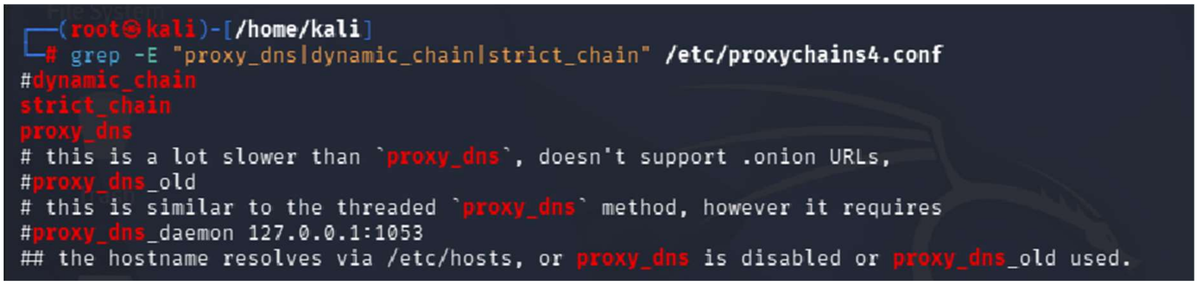
Modul 3: Layer 4 (Transport) & Layer 7 (DNS) - Rekayasa Anti-Bocor DNS

3.1. Bahaya Ancaman DNS Leak:

DNS Leak adalah insiden di mana payload data melewati jalur proxy/Tor, namun pertanyaan pencarian domain (DNS query) tetap terkirim secara telanjang (cleartext UDP port 53) ke server resolver lokal milik ISP. Hal ini mengakibatkan ISP tetap memegang riwayat lengkap daftar target yang sedang diakses.

3.2. Memeriksa Konfigurasi `proxy_dns`:

```
$ grep -E "proxy_dns|dynamic_chain|strict_chain" /etc/proxychains4.conf
```



```
┌──(root㉿kali)-[/home/kali]
└─# grep -E "proxy_dns|dynamic_chain|strict_chain" /etc/proxychains4.conf
#dynamic_chain
strict_chain
proxy_dns
# this is a lot slower than `proxy_dns`, doesn't support .onion URLs,
#proxy_dns_old
# this is similar to the threaded `proxy_dns` method, however it requires
#proxy_dns_daemon 127.0.0.1:1053
## the hostname resolves via /etc/hosts, or proxy_dns is disabled or proxy_dns_old used.
```

Screenshot 8: Verifikasi parameter proxy_dns aktif

- **proxy_dns:** Memaksa seluruh penamaan domain diselesaikan di sisi exit node proxy, kebal dari penyadapan DNS oleh ISP lokal.
- **strict_chain:** Menetapkan bahwa paket data harus melewati urutan proxy secara sekuensial dan ketat sesuai daftar di [ProxyList].

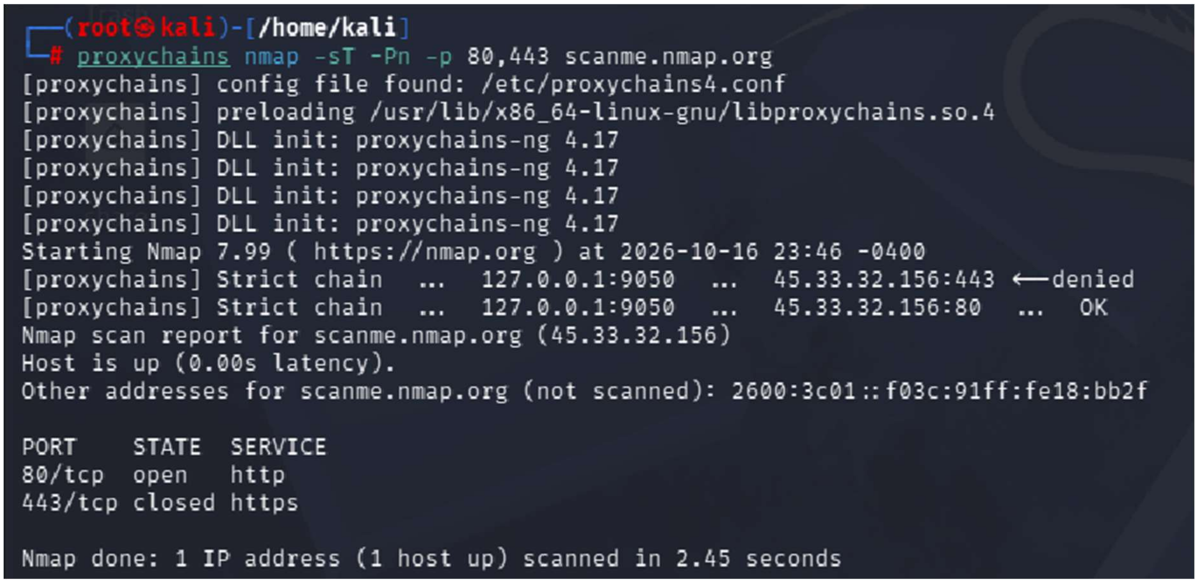
Modul 4: Port Scanning Anonim di Lapangan (`proxychains nmap`)

4.1. Aturan Emas Nmap di Balik Proxy SOCKS:

Secara default, Nmap menggunakan SYN Stealth Scan (-sS) yang membutuhkan pengiriman raw packet. Protokol SOCKS4/SOCKS5 tidak mendukung raw socket craft. Oleh karena itu, kita WAJIB menggunakan flag TCP Connect Scan (-sT) dan mematikan ICMP discovery (-Pn).

4.2. Uji Coba Scanning pada Target Resmi (scanme.nmap.org):

```
$ proxychains nmap -sT -Pn -p 80,443 scanme.nmap.org
```



```
┌──(root㉿kali)-[/home/kali]
└─# proxychains nmap -sT -Pn -p 80,443 scanme.nmap.org
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-16 23:46 -0400
[proxychains] Strict chain  ...  127.0.0.1:9050  ...  45.33.32.156:443 ⟵denied
[proxychains] Strict chain  ...  127.0.0.1:9050  ...  45.33.32.156:80  ...  OK
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.00s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f

PORT    STATE  SERVICE
80/tcp  open   http
443/tcp closed https

Nmap done: 1 IP address (1 host up) scanned in 2.45 seconds
```

Screenshot 9: Hasil nmap scan anonim via jaringan Tor

4.3. Cara Membaca Output Log:

• **45.33.32.156:443 <--denied:** Port 443 berstatus tertutup (closed), relai proxy melaporkan penolakan koneksi.

• **45.33.32.156:80 ... OK:** Port 80 berstatus terbuka (open), jabat tangan 3-way handshake TCP berhasil penuh.

• **Forensik Keamanan:** Target scanme.nmap.org hanya mencatat incoming traffic dari IP 185.220.101.110 (Jerman). Tidak ada keterkaitan ke IP asli Indonesia.

Modul 5: Kernel Layer - Transparent Proxying & Sanitasi Memori (`anonsurf`)

5.1. Keterbatasan Proxychains vs. Keunggulan Transparent Proxy:

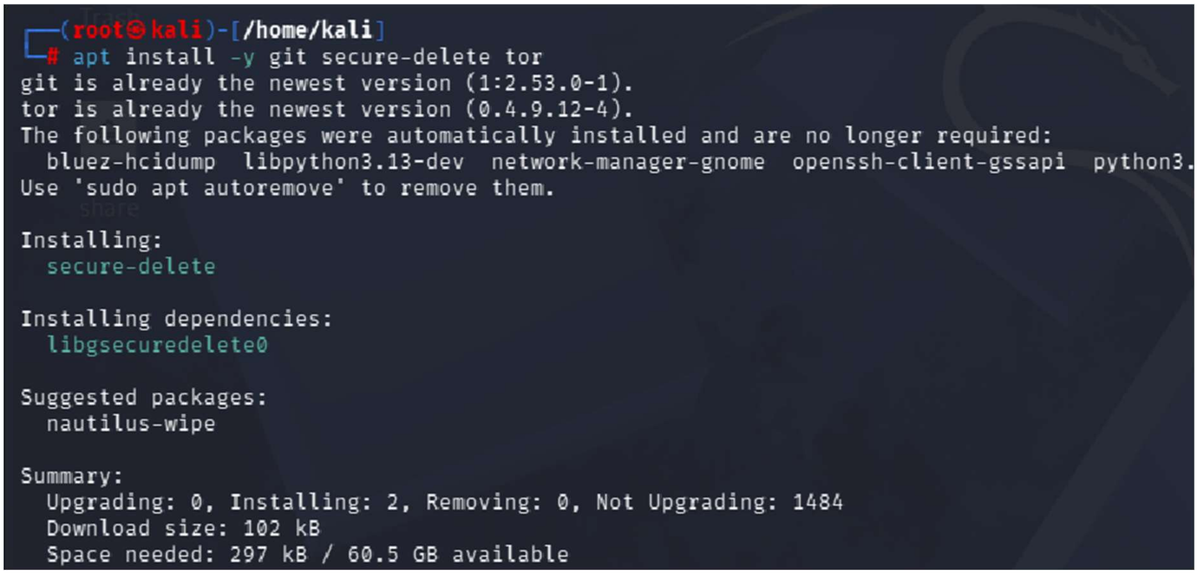
Meskipun Proxychains efektif, ia memiliki dua kelemahan mendasar: (1) Hanya mengaitkan aplikasi dinamis yang memanggil pustaka standar libc (tidak mendukung binary statis), dan (2) Rawan kelalaian manusia (human error)—jika operator lupa mengetikkan kata 'proxychains' sebelum perintah, paket akan terkirim dengan IP asli secara langsung (unprotected leak).

Solusi pamungkas untuk mengatasi masalah tersebut adalah Kernel-Level Transparent Proxying menggunakan Anonsurf. Sistem menggunakan filter paket bawaan kernel Linux (iptables) untuk mencegat SELURUH paket data TCP dan query DNS keluar (OUTPUT chain) secara transparan dan mengalihkannya langsung ke port Tor.

5.2. Langkah 9: Penyiapan Dependensi Sistem (`secure-delete` & `tor`):

Anonsurf membutuhkan paket secure-delete untuk membersihkan residu data sensitif di memori kerja (RAM) dan storage sebelum sesi anonimitas ditutup:

```
$ apt update && apt install -y secure-delete tor
```



```
┌──(root㉿kali)-[/home/kali]
└─# apt install -y git secure-delete tor
git is already the newest version (1:2.53.0-1).
tor is already the newest version (0.4.9.12-4).
The following packages were automatically installed and are no longer required:
  bluez-hcidump  libpython3.13-dev  network-manager-gnome  openssh-client-gssapi  python3.
Use 'sudo apt autoremove' to remove them.

Installing:
  secure-delete

Installing dependencies:
  libgsecuredelete0

Suggested packages:
  nautilus-wipe

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 1484
  Download size: 102 kB
  Space needed: 297 kB / 60.5 GB available
```

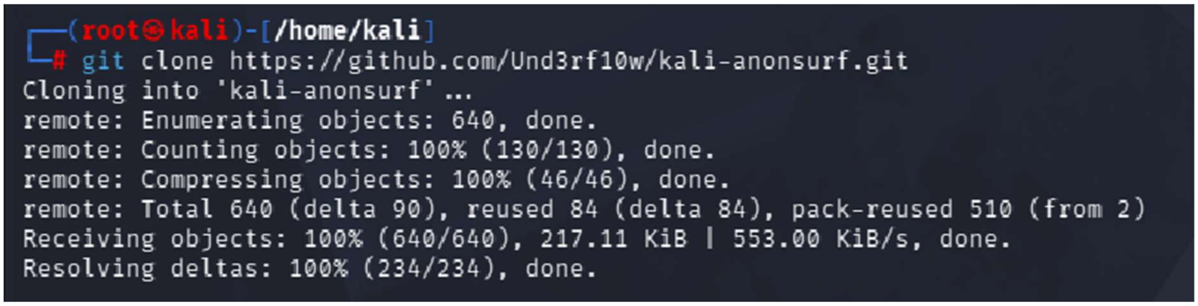
Screenshot 10: Instalasi dependensi tor dan secure-delete

• **Catatan Arsitektur:** Paket secure-delete menyediakan utilitas srm (secure remove) dan sdmem (secure memory wiper) untuk mencegah jejak forensik pada RAM fisik setelah sistem digunakan.

5.3. Langkah 10: Kloning Repositori Source Code Anonsurf:

Unduh source code resmi anonsurf dari repositori open-source:

```
$ git clone https://github.com/Und3rd0g-95/kali-anonsurf.git
```



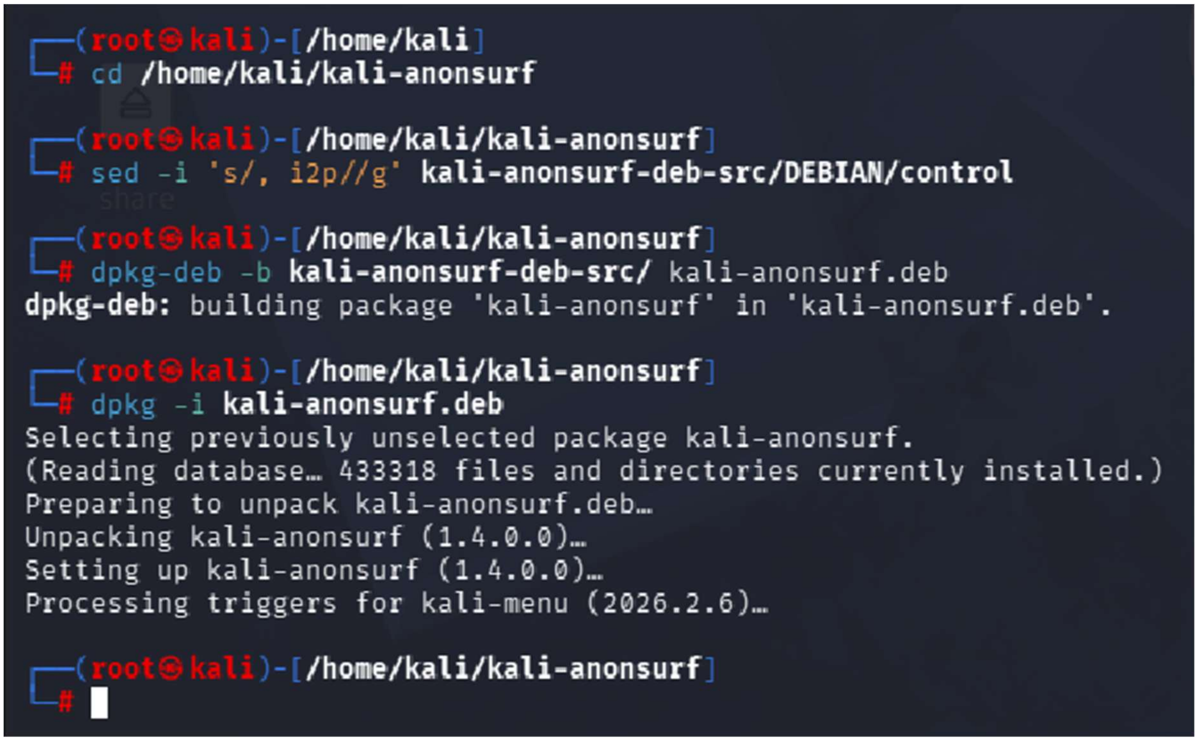
```
┌──(root㉿kali)-[/home/kali]
└─# git clone https://github.com/Und3rf10w/kali-anonsurf.git
Cloning into 'kali-anonsurf'...
remote: Enumerating objects: 640, done.
remote: Counting objects: 100% (130/130), done.
remote: Compressing objects: 100% (46/46), done.
remote: Total 640 (delta 90), reused 84 (delta 84), pack-reused 510 (from 2)
Receiving objects: 100% (640/640), 217.11 KiB | 553.00 KiB/s, done.
Resolving deltas: 100% (234/234), done.
```

Screenshot 11: Proses kloning source code kali-anonsurf ke direktori lokal

5.4. Langkah 11: Troubleshooting & Build Paket Debian (.deb):

Skrip installer bawaan sering kali mengalami kegagalan karena memanggil repositori pihak ketiga (deb.i2p.net) yang sudah usang. Untuk mengatasinya, kita menghapus dependensi i2p dari file metadata kontrol DEBIAN, lalu mengompilasi paket Debian (.deb) mandiri:

```
$ # Edit file kali-anonsurf-deb-src/DEBIAN/control (hapus dependensi i2p)
dpkg-deb -b kali-anonsurf-deb-src kali-anonsurf.deb
dpkg -i kali-anonsurf.deb
```



```
┌──(root㉿kali)-[/home/kali]
└─# cd /home/kali/kali-anonsurf

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# sed -i 's/, i2p//g' kali-anonsurf-deb-src/DEBIAN/control

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# dpkg-deb -b kali-anonsurf-deb-src/ kali-anonsurf.deb
dpkg-deb: building package 'kali-anonsurf' in 'kali-anonsurf.deb'.

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# dpkg -i kali-anonsurf.deb
Selecting previously unselected package kali-anonsurf.
(Reading database… 433318 files and directories currently installed.)
Preparing to unpack kali-anonsurf.deb…
Unpacking kali-anonsurf (1.4.0.0)…
Setting up kali-anonsurf (1.4.0.0)…
Processing triggers for kali-menu (2026.2.6)…

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# 
```

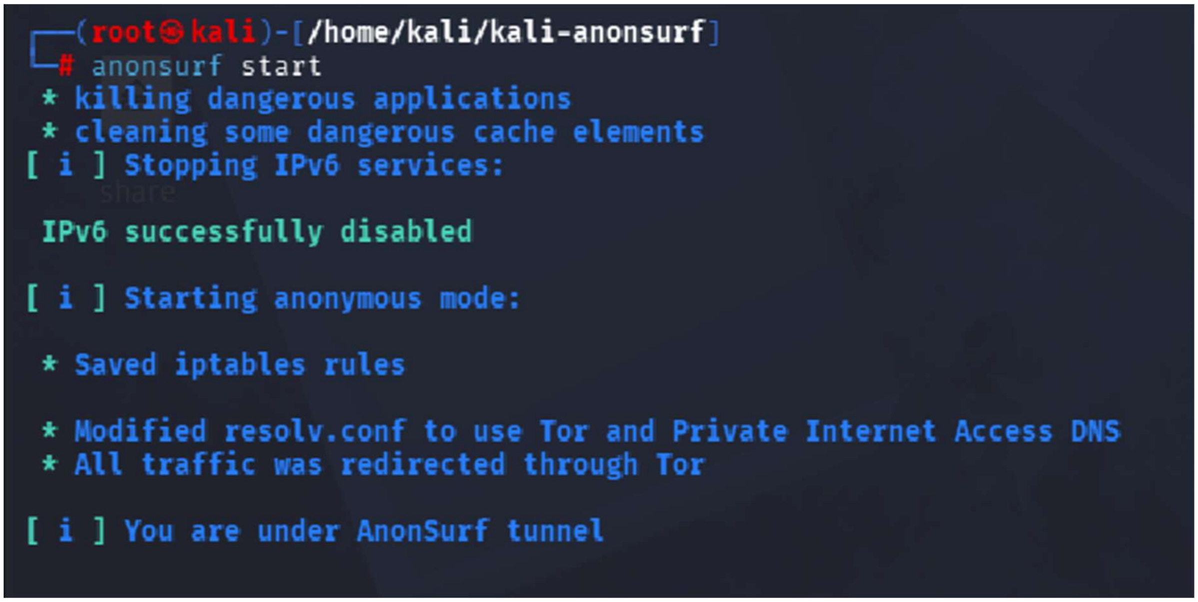
Screenshot 12: Pembuatan paket debian lokal dan instalasi kali-anonsurf sukses

📌 **TROUBLESHOOTING REPOSITORI:** Menghilangkan dependensi i2p merupakan langkah best practice pada sistem modern jika kita hanya membutuhkan fungsionalitas Tor TransPort dan DNSPort. Hal ini mencegah konflik repository yang berpotensi merusak manajemen paket APT di Kali Linux.

5.5. Langkah 12: Menjalankan Anonsurf & Mekanisme iptables:

Jalankan anonsurf untuk mengaktifkan terowongan transparan di seluruh sistem:

```
$ anonsurf start
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# anonsurf start
 * killing dangerous applications
 * cleaning some dangerous cache elements
[ i ] Stopping IPv6 services:

 IPv6 successfully disabled

[ i ] Starting anonymous mode:

 * Saved iptables rules

 * Modified resolv.conf to use Tor and Private Internet Access DNS
 * All traffic was redirected through Tor

[ i ] You are under AnonSurf tunnel
```

Screenshot 13: Menjalankan anonsurf start dan aktivasi iptables redirect

• **Mekanisme Kerja:** Saat perintah ini dipanggil, skrip anonsurf menyuntikkan konfigurasi TransPort 9040 dan DNSPort 53 ke /etc/tor/torrc, serta menerapkan tabel iptables NAT untuk membelokkan seluruh paket keluar:

1. Seluruh paket TCP keluar (kecuali milik user debian-tor dan loopback) dialihkan ke 127.0.0.1:9040.

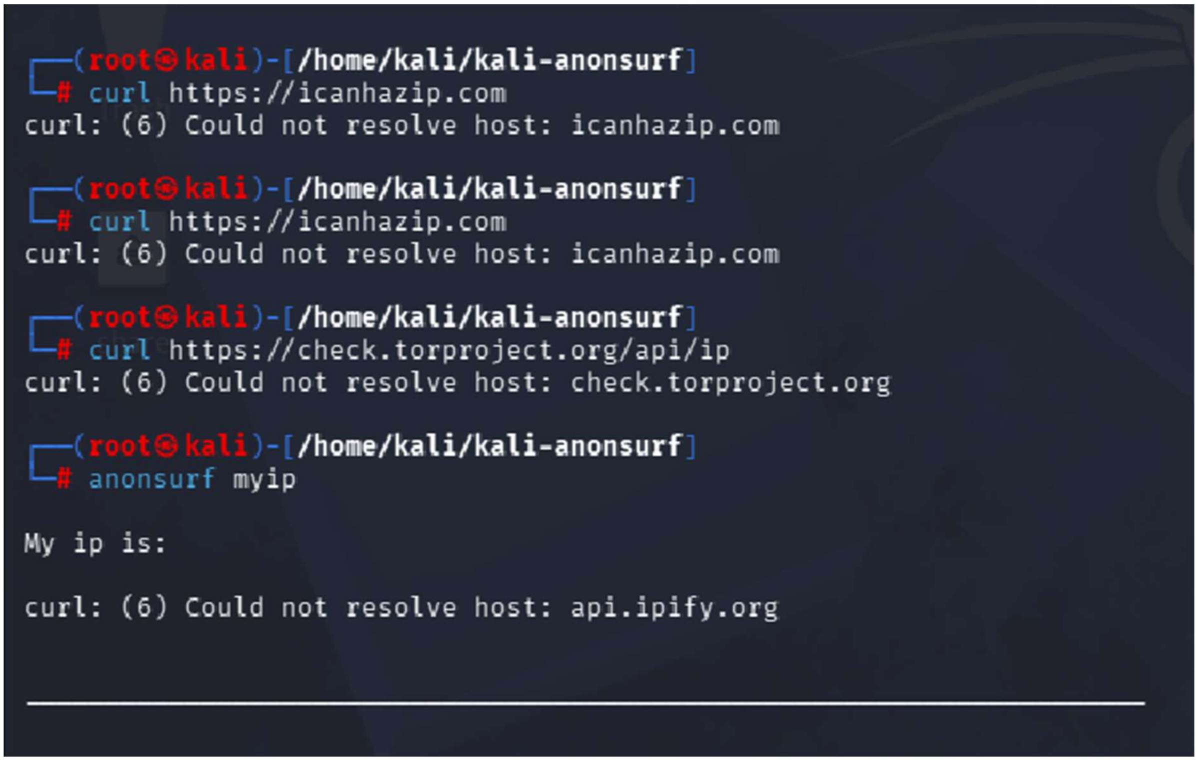
2. Seluruh kueri DNS keluar (UDP/TCP port 53) dialihkan ke 127.0.0.1:53 milik resolver internal Tor.

3. Seluruh paket non-TCP (seperti UDP/ICMP umum) diblokir (DROP) untuk mencegah IP leak tak terduga.

5.6. Langkah 13: Investigasi dan Perbaikan Resolusi DNS:

Pada inisialisasi awal, beberapa sistem mengalami galat 'Could not resolve host' karena Tor daemon belum mengikat port DNS 53 secara sempurna. Lakukan restart service untuk memperbarui socket binding:

```
$ systemctl restart tor
anonsurf status
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# curl https://icanhazip.com
curl: (6) Could not resolve host: icanhazip.com

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# curl https://icanhazip.com
curl: (6) Could not resolve host: icanhazip.com

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# curl https://check.torproject.org/api/ip
curl: (6) Could not resolve host: check.torproject.org

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# anonsurf myip

My ip is:

curl: (6) Could not resolve host: api.ipify.org
```

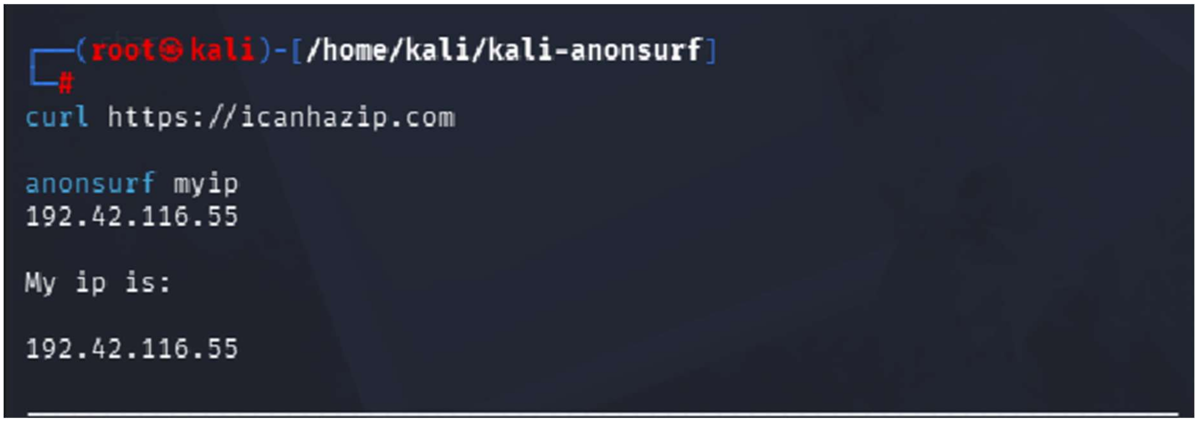
Screenshot 14: Investigasi dan perbaikan resolusi DNS pertama kali

- **Hasil Investigasi:** Setelah Tor dimuat ulang, port 53 (DNS) dan port 9040 (TransPort) aktif mendengarkan koneksi loopback, memulihkan konektivitas internet secara instan tanpa kebocoran.

5.7. Langkah 14: Pembuktian Anonimitas Tanpa Proxychains:

Uji koneksi internet menggunakan perintah biasa tanpa perlu menambahkan awalan 'proxychains':

```
$ anonsurf myip
curl https://icanhazip.com
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─#
curl https://icanhazip.com

anonsurf myip
192.42.116.55

My ip is:

192.42.116.55
```

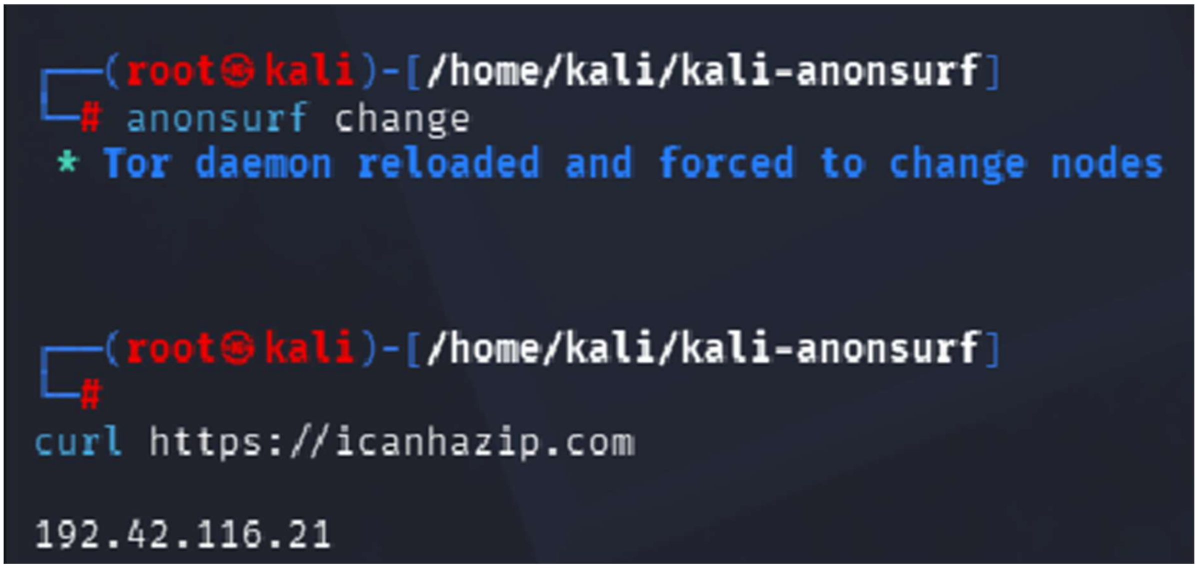
Screenshot 15: Pembuktian IP baru (192.42.116.55 - Belanda) tanpa proxychains

- **Hasil Verifikasi:** IP publik sistem terdeteksi sebagai 192.42.116.55 (Server Tor di Belanda). Browser, curl, nmap, maupun aplikasi latar belakang lainnya kini otomatis anonim secara total.

5.8. Langkah 15: Pergantian Identitas Dinamis (On-the-Fly Identity Change):

Jika IP exit node terblokir atau Anda membutuhkan identitas geografis baru di tengah operasi, ganti sirkuit Tor secara instan:

```
$ anonsurf change
anonsurf myip
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# anonsurf change
 * Tor daemon reloaded and forced to change nodes


┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─#
curl https://icanhazip.com

192.42.116.21
```

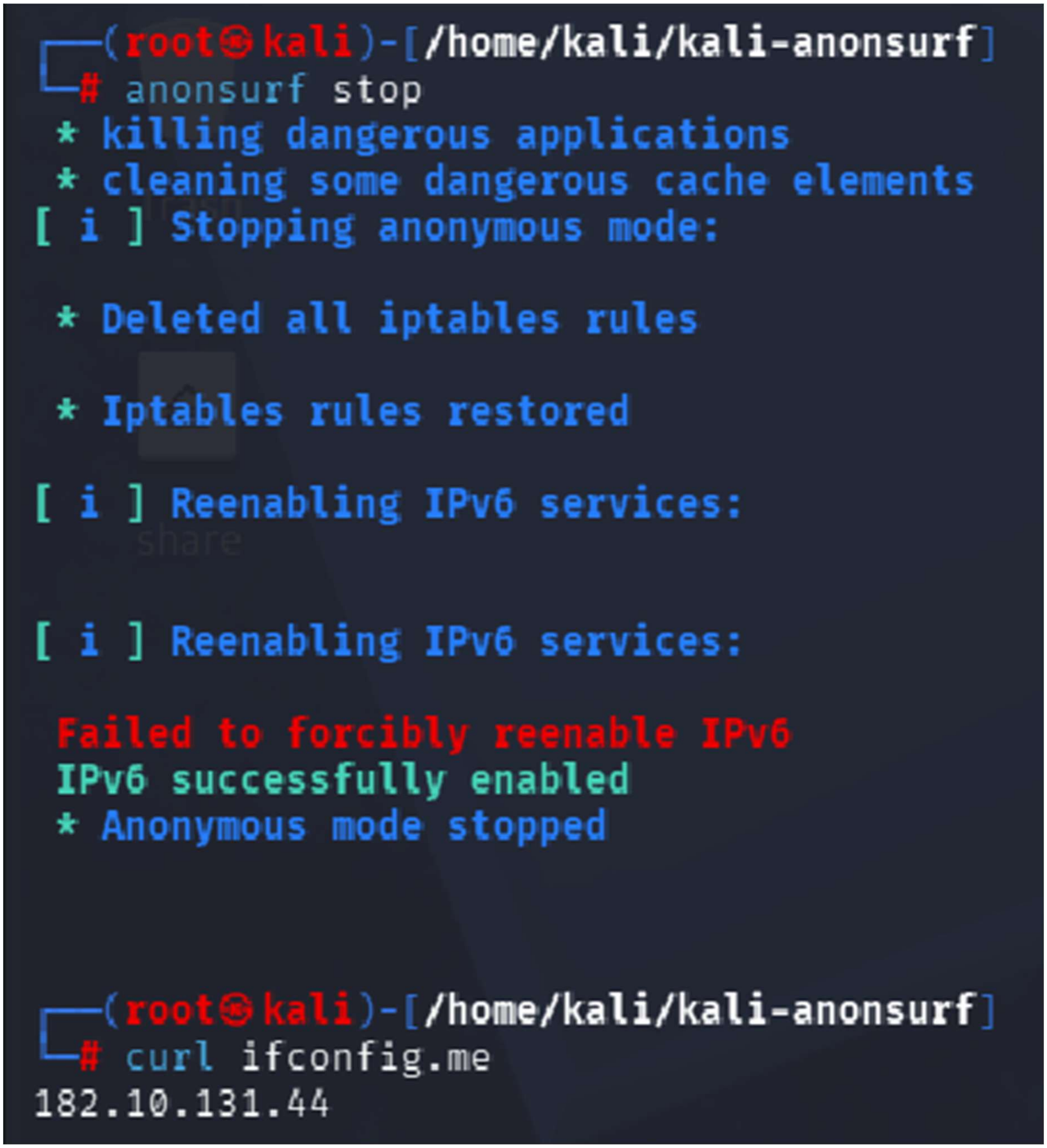
Screenshot 16: Penggantian sirkuit Tor instan ke IP baru (192.42.116.21)

• **Mekanisme Teknis:** Anonsurf mengirimkan sinyal kontrol SIGNAL NEWNYM ke port Tor Control (9051). Tor secara otomatis memutus sirkuit lama dan membangun 3 simpul baru, menghasilkan alamat IP baru (192.42.116.21) dalam hitungan detik tanpa memutus koneksi antarmuka.

5.9. Langkah 16: Penghentian Sesi, Pembersihan iptables & Sanitasi Memori:

Setelah operasi reconnaissance atau pengujian selesai, kembalikan sistem ke kondisi normal:

```
$ anonsurf stop
curl ifconfig.me
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# anonsurf stop
 * killing dangerous applications
 * cleaning some dangerous cache elements
[ i ] Stopping anonymous mode:

 * Deleted all iptables rules

 * Iptables rules restored

[ i ] Reenabling IPv6 services:


[ i ] Reenabling IPv6 services:

 Failed to forcibly reenable IPv6
 IPv6 successfully enabled
 * Anonymous mode stopped



┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# curl ifconfig.me
182.10.131.44
```

Screenshot 17: Penghentian anonsurf, pembersihan iptables, dan pemulihan IP asli

• **Analisis Pembersihan (Cleanup):** Saat anonsurf stop dijalankan, sistem secara otomatis:

1. Menghapus (flush) seluruh baris aturan pengalihan NAT pada iptables.

2. Mengembalikan file resolver /etc/resolv.conf ke konfigurasi nameserver jaringan lokal.

3. Menjalankan modul pembersih memori untuk menyapu residu cache di RAM.

• **Verifikasi Akhir:** 4. IP publik sistem secara transparan kembali ke alamat asli ISP (182.10.131.44) tanpa perlu reboot.

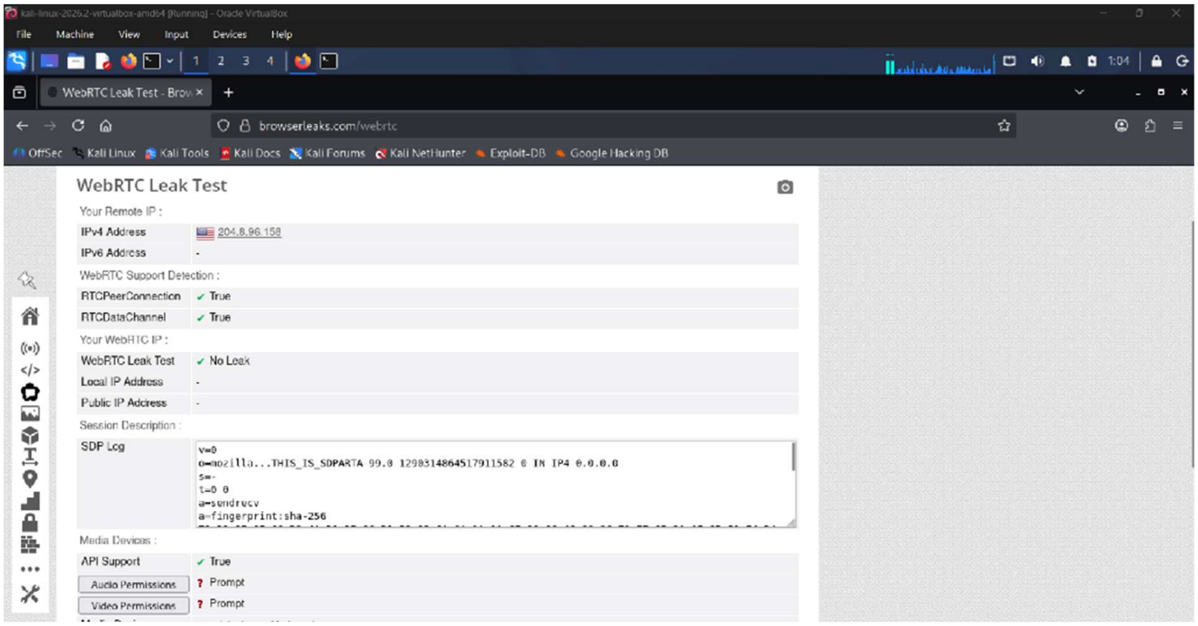
Modul 6: Application Layer - Browser Anti-Fingerprinting & Pencegahan WebRTC Leak

6.1. Celah Pelacakan Melalui Browser (WebRTC & Canvas Fingerprint):

Meskipun alamat IP terlindungi oleh Tor, peramban (browser) standar membocorkan dua vektor pelacakan berbahaya: (1) Protokol WebRTC yang menggunakan server STUN untuk mencari IP asli guna panggilan P2P melompati proxy, dan (2) Canvas Fingerprinting, yaitu teknik di mana situs target meminta browser merender teks/grafik tersembunyi. Perbedaan mikro pada arsitektur GPU dan font menghasilkan hash signature unik yang menjadi identitas permanen mesin.

6.2. Langkah 17: Uji Coba Baseline WebRTC Leak Test:

Pengujian dilakukan menggunakan browser Firefox menuju endpoint browserleaks.com/webrtc:


kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File
Machine
View
Input
Devices
Help
1:04
WebRTC Leak Test - Brow
browserleaks.com/webrtc
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB

WebRTC Leak Test

Your Remote IP :

IPv4 Address	204.8.96.158
IPv6 Address	-

WebRTC Support Detection :

RTCPeerConnection	✓ True
RTCDataChannel	✓ True

Your WebRTC IP :

WebRTC Leak Test	✓ No Leak
Local IP Address	-
Public IP Address	-

Session Description :

SDP Log

```
v=0
o=mozilla...THIS_IS_SDPARTA-99.0 1290314864517911582 0 IN IP4 0.0.0.0
s=-
t=0 0
a=sendrecv
a=fingerprint:sha-256
```

Media Devices :

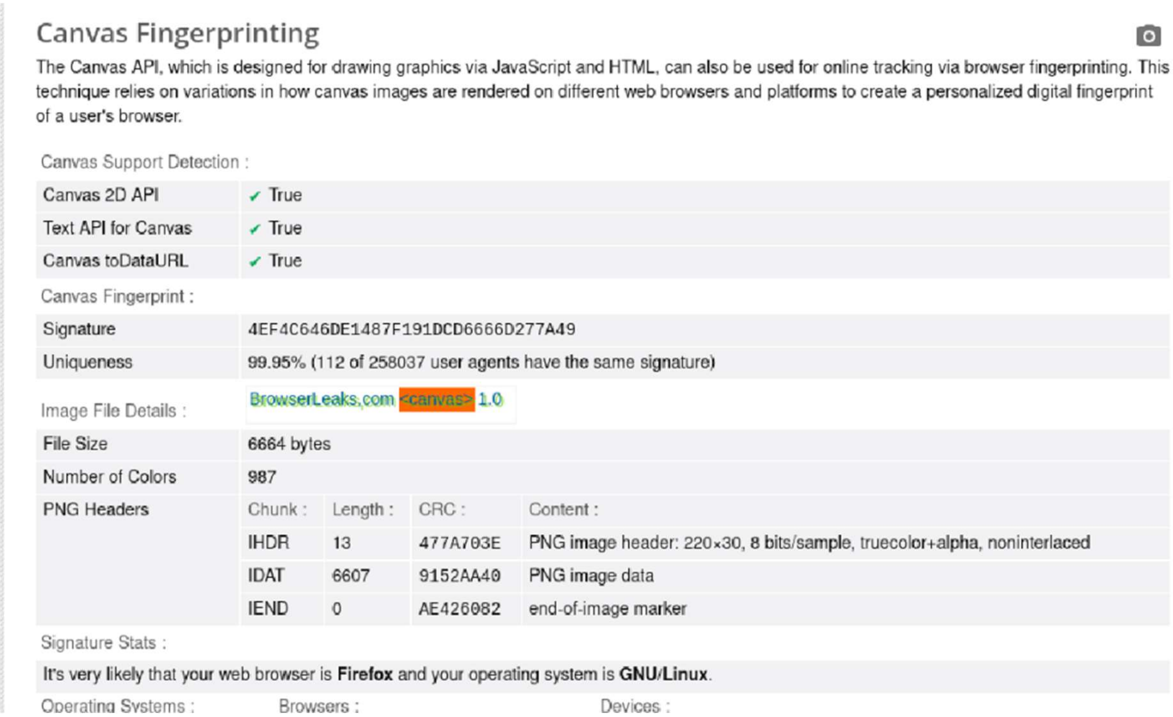
API Support	✓ True
Audio Permissions	? Prompt
Video Permissions	? Prompt

Screenshot 18: Uji baseline WebRTC - IP terlindungi Tor (204.8.96.158) dan paket STUN tertahan firewall kernel

- **Temuan Teknis:** Meskipun RTCPeerConnection berstatus True, IP asli tidak bocor karena anonsurf memblokir lalu lintas UDP di kernel. Namun WebRTC tetap berisiko jika target menggunakan STUN berbasis TCP.

6.3. Langkah 18: Pengujian Sidik Jari Kanvas (Canvas Fingerprinting Baseline):

Uji coba pembacaan sidik jari hardware melalui browserleaks.com/canvas:



Canvas Fingerprinting

The Canvas API, which is designed for drawing graphics via JavaScript and HTML, can also be used for online tracking via browser fingerprinting. This technique relies on variations in how canvas images are rendered on different web browsers and platforms to create a personalized digital fingerprint of a user's browser.

Canvas Support Detection :

Canvas 2D API	✓ True
Text API for Canvas	✓ True
Canvas toDataURL	✓ True

Canvas Fingerprint :

Signature	4EF4C646DE1487F191DCD6666D277A49
Uniqueness	99.95% (112 of 258037 user agents have the same signature)

Image File Details : BrowserLeaks.com <canvas> 1.0

File Size	6664 bytes
Number of Colors	987

PNG Headers

Chunk :	Length :	CRC :	Content :
IHDR	13	477A703E	PNG image header: 220×30, 8 bits/sample, truecolor+alpha, noninterlaced
IDAT	6607	9152AA40	PNG image data
IEND	0	AE426082	end-of-image marker

Signature Stats :

It's very likely that your web browser is **Firefox** and your operating system is **GNU/Linux**.

Operating Systems : Browsers : Devices :

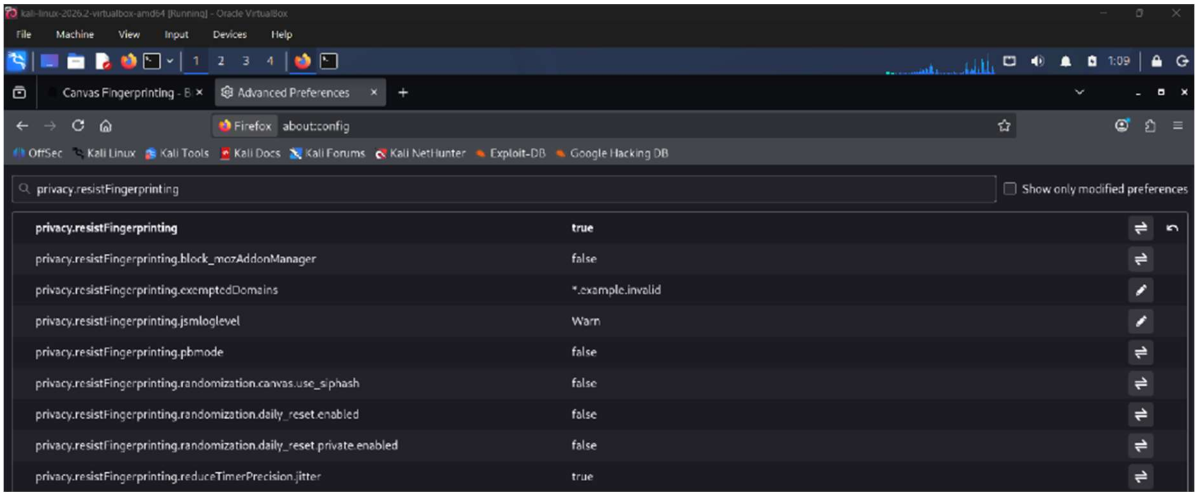
Screenshot 19: Pengujian Canvas Fingerprinting awal menunjukkan signature unik 4EF4C646... dengan keunikan 99.95%

- **Bahaya Forensik:** Hasil sidik jari kanvas terbaca sangat spesifik (Signature: 4EF4C646DE1487F191DCD6666D277A49, 987 warna, ukuran 6664 bytes). Sistem analitik dapat mendeteksi persis bahwa sistem adalah Firefox pada GNU/Linux.

6.4. Langkah 19: Hardening Firefox Melalui about:config:

Mengaktifkan modul perlindungan bawaan Tor Project pada Firefox untuk mematikan WebRTC dan meracuni sidik jari kanvas:

```
$ # Pada address bar Firefox, buka: about:config
media.peerconnection.enabled = false
privacy.resistFingerprinting = true
```

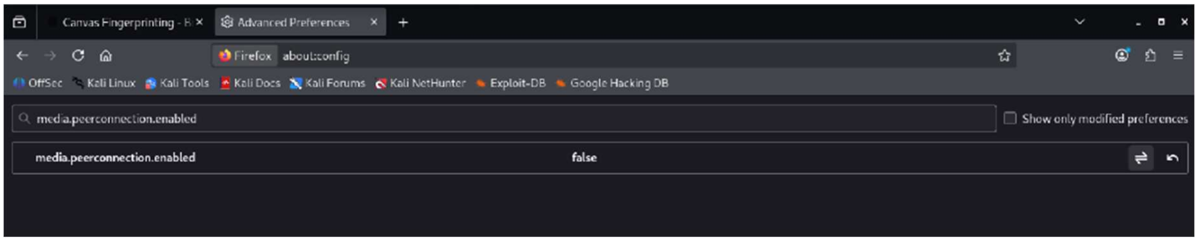

kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File
Machine
View
Input
Devices
Help
1:09
Canvas Fingerprinting - B
Advanced Preferences
Firefox
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
☐ Show only modified preferences

about:config

privacy.resistFingerprinting

privacy.resistFingerprinting	**true**
privacy.resistFingerprinting.block_mozAddonManager	false
privacy.resistFingerprinting.exemptedDomains	*.example.invalid
privacy.resistFingerprinting.jsmloglevel	Warn
privacy.resistFingerprinting.pbmode	false
privacy.resistFingerprinting.randomization.canvas.use_siphash	false
privacy.resistFingerprinting.randomization.daily_reset.enabled	false
privacy.resistFingerprinting.randomization.daily_reset.private.enabled	false
privacy.resistFingerprinting.reduceTimerPrecision.jitter	true

Screenshot 20: Aktivasi modul anti-pelacakan privacy.resistFingerprinting = true


Canvas Fingerprinting - B
Advanced Preferences
Firefox
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
☐ Show only modified preferences

about:config

media.peerconnection.enabled

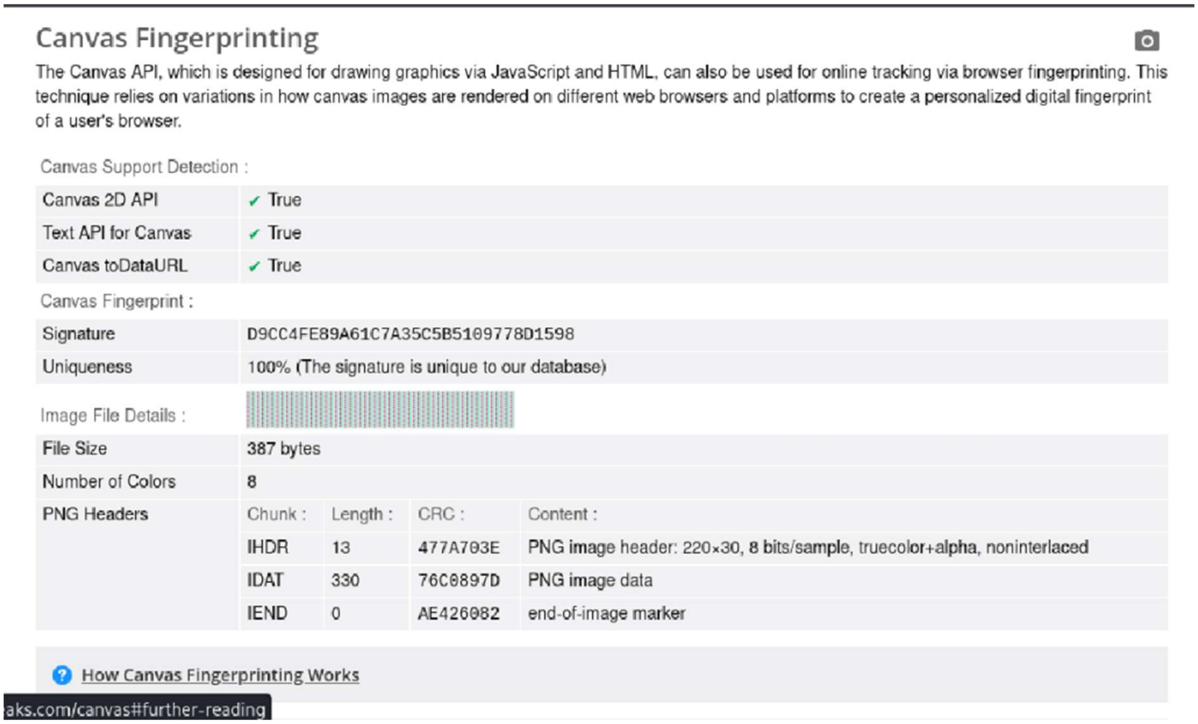
media.peerconnection.enabled	**false**

Screenshot 21: Menonaktifkan subsistem WebRTC media.peerconnection.enabled = false

- **Analisis Hardening:** Modul resistFingerprinting secara otomatis membulatkan resolusi layar ke dimensi standar, menyamarkan zona waktu sistem ke UTC, menyamarkan User-Agent, dan menyuntikkan noise acak pada setiap permintaan rendering canvas.

6.5. Langkah 20: Verifikasi Hasil Hardening (Canvas Spoofing Terbukti):

Setelah halaman canvas di-reload, browser secara aktif meracuni hasil pembacaan sidik jari:



Canvas Fingerprinting

The Canvas API, which is designed for drawing graphics via JavaScript and HTML, can also be used for online tracking via browser fingerprinting. This technique relies on variations in how canvas images are rendered on different web browsers and platforms to create a personalized digital fingerprint of a user's browser.

Canvas Support Detection :

Canvas 2D API	✓ True
Text API for Canvas	✓ True
Canvas toDataURL	✓ True

Canvas Fingerprint :

Signature	D9CC4FE89A61C7A35C5B5109778D1598
Uniqueness	100% (The signature is unique to our database)

Image File Details :

File Size	387 bytes			
Number of Colors	8			
PNG Headers	Chunk :	Length :	CRC :	Content :
	IHDR	13	477A703E	PNG image header: 220×30, 8 bits/sample, truecolor+alpha, noninterlaced
	IDAT	330	76C0897D	PNG image data
	IEND	0	AE426082	end-of-image marker

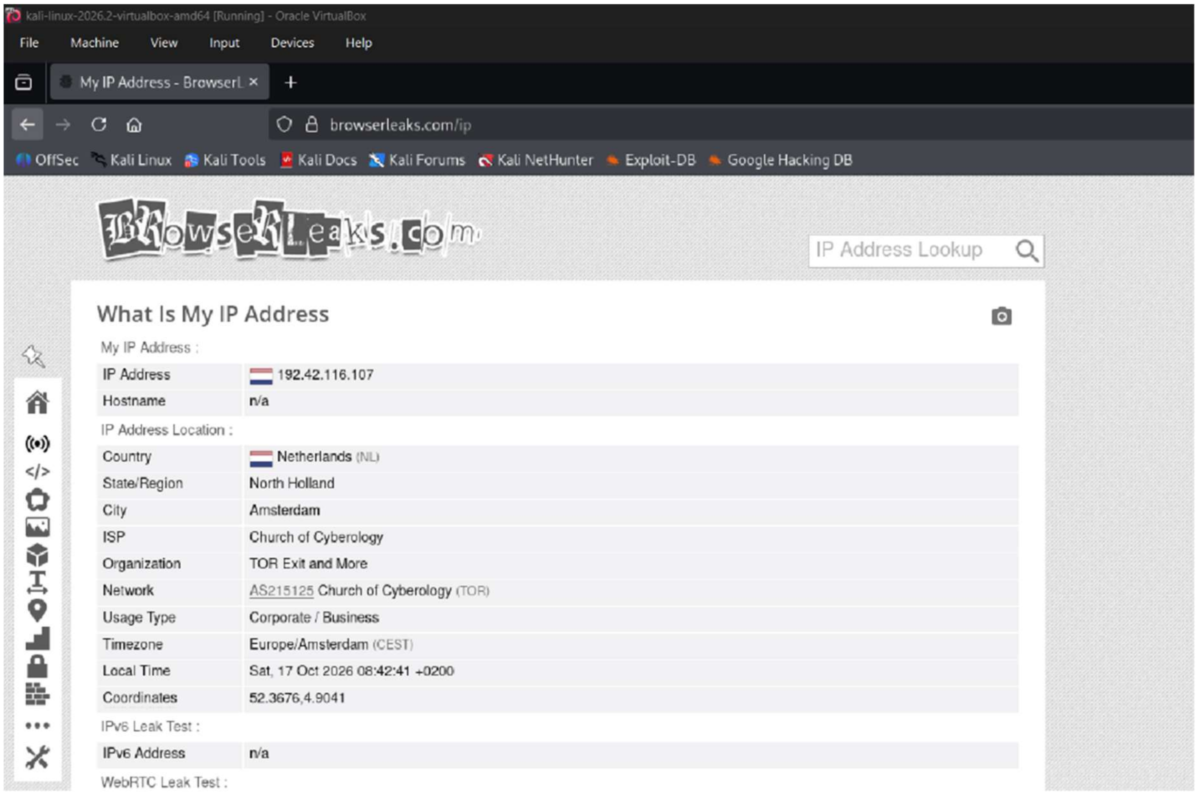
How Canvas Fingerprinting Works
aks.com/canvas#further-reading

Screenshot 22: Bukti keberhasilan - Signature teracak ke D9CC4FE8..., warna turun ke 8 warna, dan deteksi OS buta

- **Bukti Efektivitas:** Ukuran file rendering turun drastis dari 6664 bytes ke 387 bytes, jumlah warna terpangkas dari 987 menjadi hanya 8 warna berupa pola noise terdistorsi. Signature berubah total dan algoritma penilai sidik jari gagal memprofiling identitas laptop.

6.6. Langkah Tambahan: Verifikasi Alamat IP & Lokasi di Browser:

Pemeriksaan menyeluruh pada endpoint browserleaks.com/ip memastikan bahwa seluruh komponen browser kini terdeteksi bersih sebagai Tor Exit Node:


kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File
Machine
View
Input
Devices
Help
My IP Address - BrowserL
browserleaks.com/ip
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
BrowserLeaks.com
IP Address Lookup

What Is My IP Address

My IP Address :

IP Address	192.42.116.107
Hostname	n/a

IP Address Location :

Country	Netherlands (NL)
State/Region	North Holland
City	Amsterdam
ISP	Church of Cyberology
Organization	TOR Exit and More
Network	AS215125 Church of Cyberology (TOR)
Usage Type	Corporate / Business
Timezone	Europe/Amsterdam (CEST)
Local Time	Sat, 17 Oct 2026 08:42:41 +0200
Coordinates	52.3676,4.9041

IPv6 Leak Test :

IPv6 Address	n/a

WebRTC Leak Test :

Screenshot 23: BrowserLeaks IP Test mendeteksi IP 192.42.116.107 (Tor Exit Node Amsterdam, Belanda) tanpa kebocoran IPv6 atau WebRTC

Modul 7: Data & Metadata Layer - Sanitasi Metadata File Digital (`mat2`)

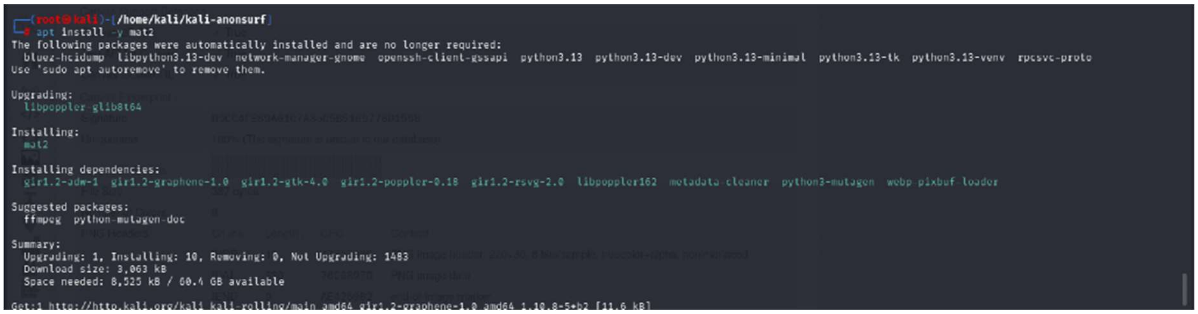
7.1. Bahaya Residu Metadata File Digital:

Setiap file digital (gambar JPEG/PNG, dokumen Word DOCX, atau PDF) menyematkan metadata tersembunyi (EXIF tags) yang mencatat: nama pembuat (Author/Artist), merk kamera/ponsel, software pengolah grafis, hingga koordinat GPS (latitude & longitude). Mengunggah file dengan metadata utuh akan membocorkan identitas fisik pelaku meskipun transmisi data dilakukan via Tor.

7.2. Langkah 21: Instalasi Toolkit Sanitasi `mat2`:

Instalasi Metadata Anonymisation Toolkit v2 pada terminal Kali Linux:

```
$ apt install -y mat2
```



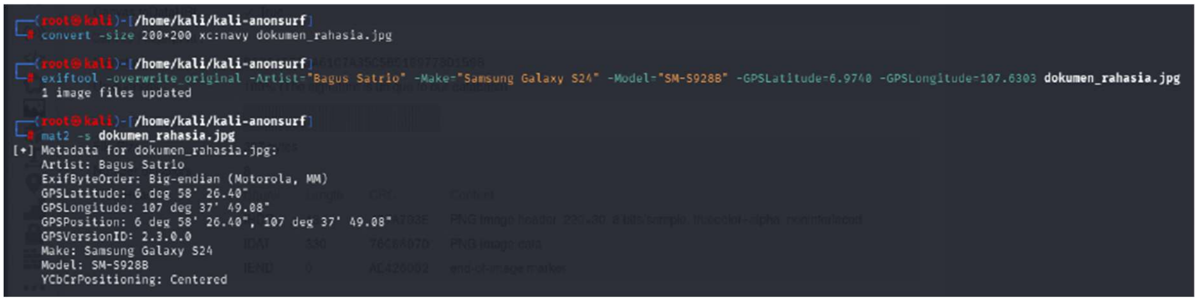
```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# apt install -y mat2
The following packages were automatically installed and are no longer required:
  bluez-hcidump  libpython3.13-dev  network-manager-gnome  openssh-client-gssapi  python3.13  python3.13-dev  python3.13-minimal  python3.13-tk  python3.13-venv  rpcsvc-proto
Use 'sudo apt autoremove' to remove them.

Upgrading:
  libpoppler-glib8t64

Installing:
  mat2

Installing dependencies:
  gir1.2-adw-1  gir1.2-graphene-1.0  gir1.2-gtk-4.0  gir1.2-poppler-0.18  gir1.2-rsvg-2.0  libpoppler162  metadata-cleaner  python3-mutagen  webp-pixbuf-loader

Suggested packages:
  ffmpeg  python-mutagen-doc

Summary:
  Upgrading: 1, Installing: 10, Removing: 0, Not Upgrading: 1483
  Download size: 3,063 kB
  Space needed: 8,525 kB / 60.4 GB available
Get:1 http://http.kali.org/kali kali-rolling/main amd64 gir1.2-graphene-1.0 amd64 1.10.8-5+b2 [11.6 kB]
```

Screenshot 24: Proses instalasi toolkit mat2 dan dependensi library pendukung

7.3. Langkah 22: Eksperimen Membongkar Metadata Tersembunyi (Forensic Inspection):

Simulasi file sampel yang disuntikkan metadata identitas, perangkat, dan koordinat GPS:

```
$ convert -size 200x200 xc:navy dokumen_rahasia.jpg
exiftool -overwrite_original -Artist="Bagus Satrio" -Make="Samsung Galaxy S24" -
Model="SM-S928B" -GPSLatitude=6.9740 -GPSLongitude=107.6303 dokumen_rahasia.jpg
mat2 -s dokumen_rahasia.jpg
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# convert -size 200×200 xc:navy dokumen_rahasia.jpg

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# exiftool -overwrite_original -Artist="Bagus Satrio" -Make="Samsung Galaxy S24" -Model="SM-S928B" -GPSLatitude=6.9740 -GPSLongitude=107.6303 dokumen_rahasia.jpg
    1 image files updated

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# mat2 -s dokumen_rahasia.jpg
[+] Metadata for dokumen_rahasia.jpg:
    Artist: Bagus Satrio
    ExifByteOrder: Big-endian (Motorola, MM)
    GPSLatitude: 6 deg 58' 26.40"
    GPSLongitude: 107 deg 37' 49.08"
    GPSPosition: 6 deg 58' 26.40", 107 deg 37' 49.08"
    GPSVersionID: 2.3.0.0
    Make: Samsung Galaxy S24
    Model: SM-S928B
    YCbCrPositioning: Centered
```

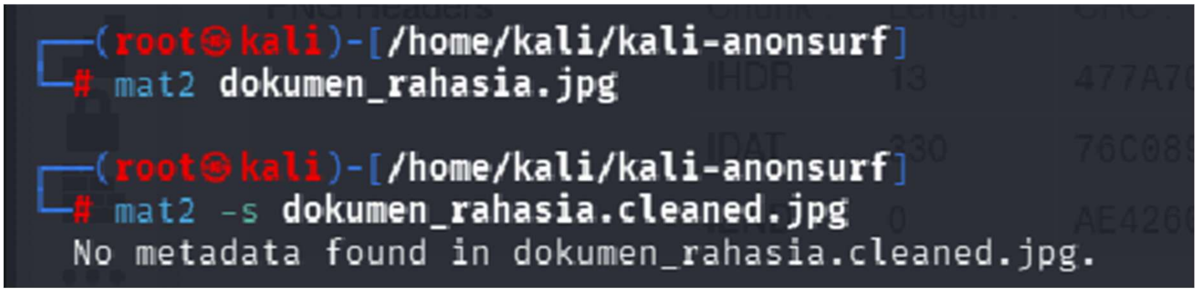
Screenshot 25: mat2 membongkar metadata rahasia: nama Bagus Satrio, model HP, dan koordinat GPS persis

- **Bahaya Kebocoran:** Seluruh identitas operator (Nama: Bagus Satrio, Model: Samsung Galaxy S24, Posisi GPS: 6°58'26.40" S, 107°37'49.08" E) terpampang telanjang jika file ini tidak dibersihkan.

7.4. Langkah 23: Eksekusi Sanitasi Total & Verifikasi Zero-Footprint:

Pembersihan metadata file menggunakan mat2 dan pembuktian status sterilisasi:

```
$ mat2 dokumen_rahasia.jpg
mat2 -s dokumen_rahasia.cleaned.jpg
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# mat2 dokumen_rahasia.jpg

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# mat2 -s dokumen_rahasia.cleaned.jpg
    No metadata found in dokumen_rahasia.cleaned.jpg.
```

Screenshot 26: Hasil verifikasi file bersih menyatakan: No metadata found in dokumen_rahasia.cleaned.jpg

- **Analisis Keamanan:** mat2 merekonstruksi ulang data mentah file tanpa membawa struktur tag EXIF. Status 'No metadata found' membuktikan file telah 100% aman dan bebas dari jejak identitas personal sebelum didistribusikan ke publik.

Modul 8: DPI Obfuscation - Tor Pluggable Transports & Studi Kasus Protocol Deadlock

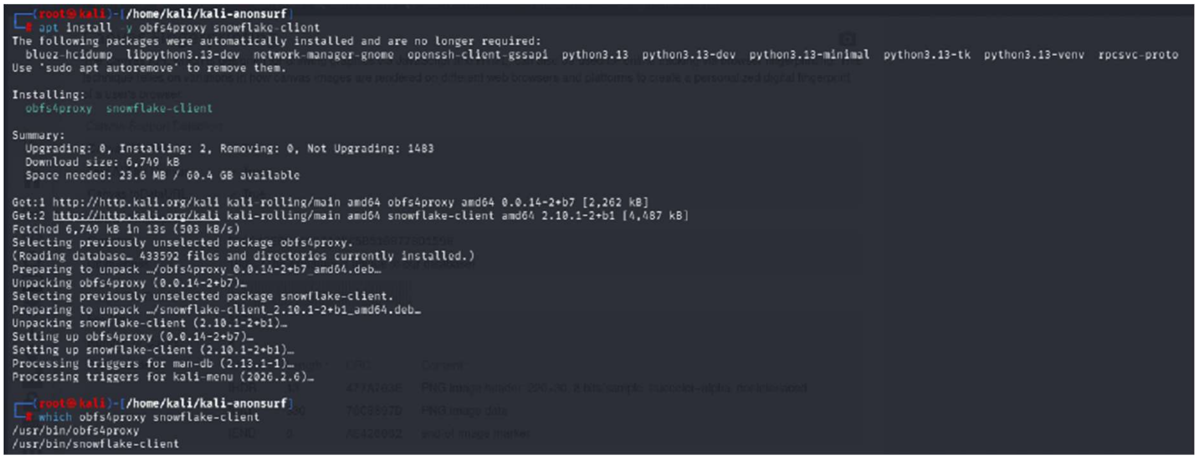
8.1. Konsep & Celah Pengawasan ISP (Deep Packet Inspection):

Daftar IP seluruh Guard Node Tor bersifat publik. ISP dan badan sensor pemerintah dapat menggunakan modul Deep Packet Inspection (DPI) untuk mengenali jabat tangan TLS Tor dan mencatat bahwa pelanggan tertentu sedang menggunakan jaringan gelap (Darknet). Untuk mengatasinya, dikembangkan teknologi Tor Bridges (relai privat tak terdaftar) dan Pluggable Transports (pengubah bentuk lalu lintas data).

8.2. Langkah 24: Instalasi Paket Pluggable Transports (`obfs4proxy` & `snowflake-client`):

Memasang utilitas pembungkus paket di terminal Kali Linux:

```
$ apt install -y obfs4proxy snowflake-client
which obfs4proxy snowflake-client
```



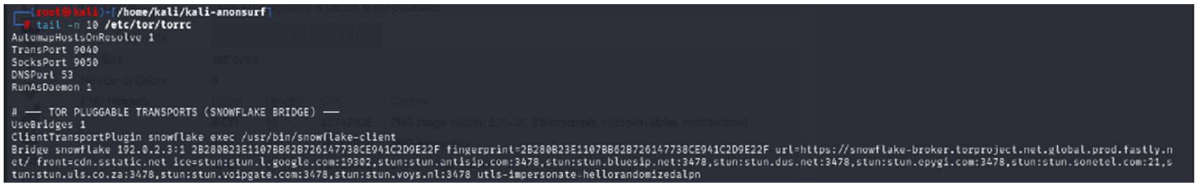
```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# apt install -y obfs4proxy snowflake-client
The following packages were automatically installed and are no longer required:
  bluez-hcidump  libpython3.13-dev  network-manager-gnome  openssh-client-gssapi  python3.13  python3.13-dev  python3.13-minimal  python3.13-tk  python3.13-venv  rpcsvc-proto
Use 'sudo apt autoremove' to remove them.

Installing:
  obfs4proxy  snowflake-client

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 1483
  Download size: 6,749 kB
  Space needed: 23.6 MB / 60.4 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 obfs4proxy amd64 0.0.14-2+b7 [2,262 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 snowflake-client amd64 2.10.1-2+b1 [4,487 kB]
Fetched 6,749 kB in 13s (503 kB/s)
Selecting previously unselected package obfs4proxy.
(Reading database… 433592 files and directories currently installed.)
Preparing to unpack …/obfs4proxy_0.0.14-2+b7_amd64.deb…
Unpacking obfs4proxy (0.0.14-2+b7)…
Selecting previously unselected package snowflake-client.
Preparing to unpack …/snowflake-client_2.10.1-2+b1_amd64.deb…
Unpacking snowflake-client (2.10.1-2+b1)…
Setting up obfs4proxy (0.0.14-2+b7)…
Setting up snowflake-client (2.10.1-2+b1)…
Processing triggers for man-db (2.13.1-1)…
Processing triggers for kali-menu (2026.2.6)…

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# which obfs4proxy snowflake-client
/usr/bin/obfs4proxy
/usr/bin/snowflake-client
```

Screenshot 27: Instalasi dan verifikasi binary obfs4proxy dan snowflake-client di /usr/bin/

8.3. Langkah 25: Konfigurasi Bridge Snowflake pada `/etc/tor/torrc`:

Menyuntikkan baris jembatan Snowflake resmi yang memanfaatkan Domain Fronting (Fastly CDN & Google STUN):

```
$ UseBridges 1
ClientTransportPlugin snowflake exec /usr/bin/snowflake-client
Bridge snowflake 192.0.2.3:1 2B280B23E1107BB62B726147738CE941C2D9E22F ...
```



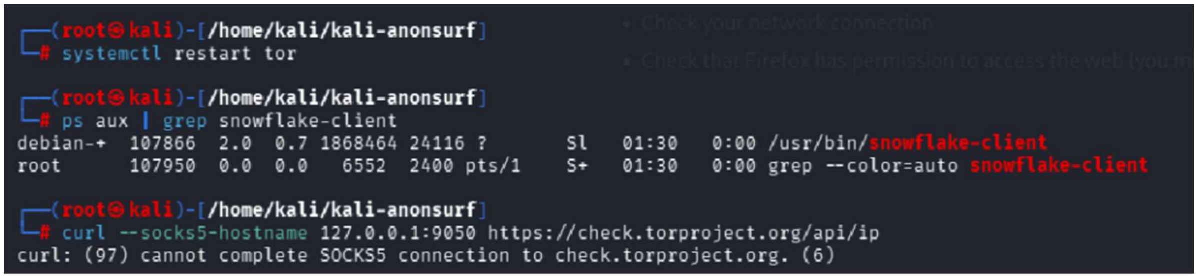
```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# tail -n 10 /etc/tor/torrc
AutomapHostsOnResolve 1
TransPort 9040
SocksPort 9050
DNSPort 53
RunAsDaemon 1

# ── TOR PLUGGABLE TRANSPORTS (SNOWFLAKE BRIDGE) ──
UseBridges 1
ClientTransportPlugin snowflake exec /usr/bin/snowflake-client
Bridge snowflake 192.0.2.3:1 2B280B23E1107BB62B726147738CE941C2D9E22F fingerprint=2B280B23E1107BB62B726147738CE941C2D9E22F url=https://snowflake-broker.torproject.net.global.prod.fastly.n
et/ front=cdn.sstatic.net ice=stun:stun.l.google.com:19302,stun:stun.antisip.com:3478,stun:stun.bluesip.net:3478,stun:stun.dus.net:3478,stun:stun.epygi.com:3478,stun:stun.sonetel.com:21,s
tun:stun.uls.co.za:3478,stun:stun.voipgate.com:3478,stun:stun.voys.nl:3478 utls-impersonate-hellorandomizedalpn
```

Screenshot 28: Konfigurasi jembatan Snowflake pada file /etc/tor/torrc

8.4. Langkah 26: Eksekusi Proses Snowflake & Investigasi Koneksi:

Saat Tor dinyalakan ulang, proses snowflake-client berhasil dipanggil sebagai sub-proses resmi oleh user debian-tor:

```
$ systemctl restart tor
ps aux | grep snowflake-client
```



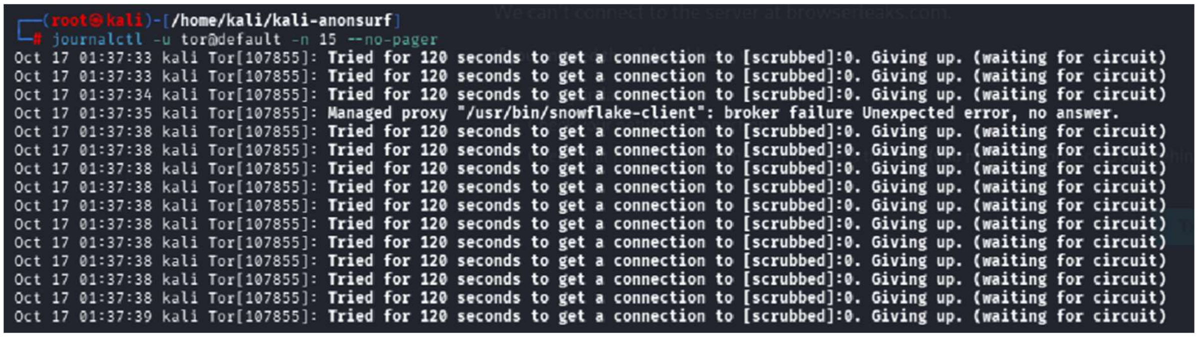
```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# systemctl restart tor

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# ps aux | grep snowflake-client
debian-+  107866  2.0  0.7 1868464 24116 ?       Sl   01:30   0:00 /usr/bin/snowflake-client
root      107950  0.0  0.0   6552  2400 pts/1    S+   01:30   0:00 grep --color=auto snowflake-client

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# curl --socks5-hostname 127.0.0.1:9050 https://check.torproject.org/api/ip
curl: (97) cannot complete SOCKS5 connection to check.torproject.org. (6)
```

Screenshot 29: Verifikasi proses snowflake-client aktif berjalan di PID 107866

8.5. Langkah 27: Investigasi Benturan Arsitektur (Troubleshooting Deadlock):

Pada pengujian lapangan, koneksi mengalami perlambatan (hang). Melalui analisis log terungkap penyebab benturan arsitektur:

```
$ journalctl -u tor@default -n 15 --no-pager
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# journalctl -u tor@default -n 15 --no-pager
Oct 17 01:37:33 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:33 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:34 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:35 kali Tor[107855]: Managed proxy "/usr/bin/snowflake-client": broker failure Unexpected error, no answer.
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:38 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
Oct 17 01:37:39 kali Tor[107855]: Tried for 120 seconds to get a connection to [scrubbed]:0. Giving up. (waiting for circuit)
```

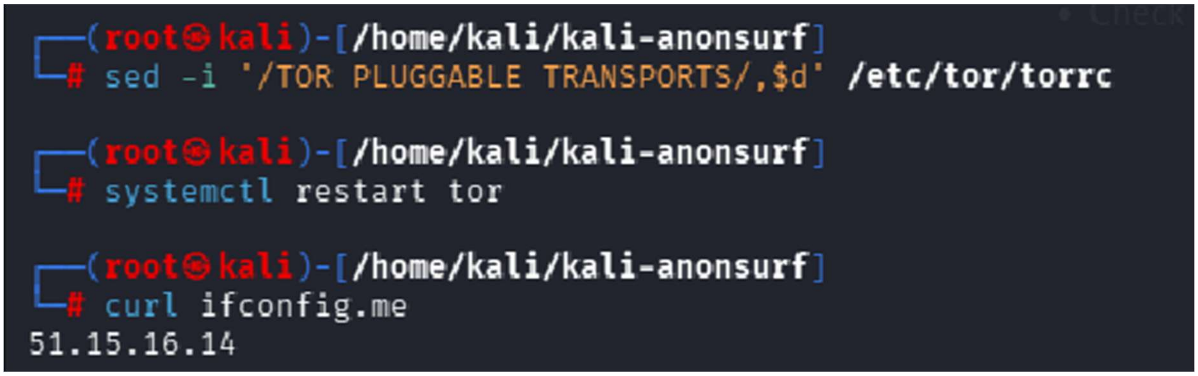
Screenshot 30: Log forensik menunjukkan broker failure akibat pemblokiran UDP oleh iptables

📌 **ANALISIS PROTOCOL DEADLOCK:** Anonsurf didesain untuk memblokir seluruh paket non-TCP (DROP UDP) demi mencegah kebocoran IP. Sementara itu, Snowflake bekerja menggunakan WebRTC yang membutuhkan paket UDP ke server STUN. Akibatnya, iptables Anonsurf mencekik koneksi negosiasi Snowflake sendiri (deadlock). Ini adalah studi kasus nyata dalam rekayasa jaringan pertahanan.

8.6. Langkah 28: Pemulihan Konfigurasi & Restorasi Kecepatan Maksimal:

Mengembalikan konfigurasi Tor ke mode standar berkecepatan tinggi dengan tetap mempertahankan seluruh lapisan perlindungan Anonsurf:

```
$ sed -i '/TOR PLUGGABLE TRANSPORTS/,$d' /etc/tor/torrc
systemctl restart tor
curl ifconfig.me
```



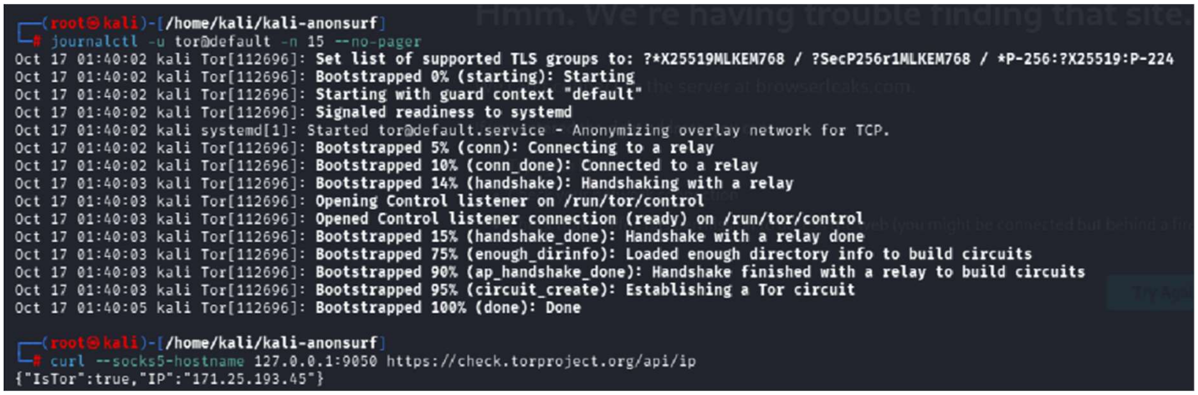
```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# sed -i '/TOR PLUGGABLE TRANSPORTS/,$d' /etc/tor/torrc

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# systemctl restart tor

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# curl ifconfig.me
51.15.16.14
```

Screenshot 31: Kecepatan pulih instan dengan IP Exit Node baru (51.15.16.14 - Prancis)

8.7. Langkah 29: Pembuktian Bootstrapping 100% & Validasi Status IsTor: true:

Pemeriksaan log siklus jabat tangan sirkuit Tor membuktikan keberhasilan bootstrapping 100% tuntas dalam hitungan 3 detik:

```
$ journalctl -u tor@default -n 15 --no-pager
curl --socks5-hostname 127.0.0.1:9050 https://check.torproject.org/api/ip
```



```
┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# journalctl -u tor@default -n 15 --no-pager
Oct 17 01:40:02 kali Tor[112696]: Set list of supported TLS groups to: ?*X25519MLKEM768 / ?SecP256r1MLKEM768 / *P-256:?X25519:P-224
Oct 17 01:40:02 kali Tor[112696]: Bootstrapped 0% (starting): Starting
Oct 17 01:40:02 kali Tor[112696]: Starting with guard context "default"
Oct 17 01:40:02 kali Tor[112696]: Signaled readiness to systemd
Oct 17 01:40:02 kali systemd[1]: Started tor@default.service - Anonymizing overlay network for TCP.
Oct 17 01:40:02 kali Tor[112696]: Bootstrapped 5% (conn): Connecting to a relay
Oct 17 01:40:02 kali Tor[112696]: Bootstrapped 10% (conn_done): Connected to a relay
Oct 17 01:40:03 kali Tor[112696]: Bootstrapped 14% (handshake): Handshaking with a relay
Oct 17 01:40:03 kali Tor[112696]: Opening Control listener on /run/tor/control
Oct 17 01:40:03 kali Tor[112696]: Opened Control listener connection (ready) on /run/tor/control
Oct 17 01:40:03 kali Tor[112696]: Bootstrapped 15% (handshake_done): Handshake with a relay done
Oct 17 01:40:03 kali Tor[112696]: Bootstrapped 75% (enough_dirinfo): Loaded enough directory info to build circuits
Oct 17 01:40:03 kali Tor[112696]: Bootstrapped 90% (ap_handshake_done): Handshake finished with a relay to build circuits
Oct 17 01:40:03 kali Tor[112696]: Bootstrapped 95% (circuit_create): Establishing a Tor circuit
Oct 17 01:40:05 kali Tor[112696]: Bootstrapped 100% (done): Done

┌──(root㉿kali)-[/home/kali/kali-anonsurf]
└─# curl --socks5-hostname 127.0.0.1:9050 https://check.torproject.org/api/ip
{"IsTor":true,"IP":"171.25.193.45"}
```

Screenshot 32: Bukti proses bootstrapping 100% tuntas dalam 3 detik dan verifikasi respon resmi IsTor: true dengan IP 171.25.193.45

- **Analisis Siklus Sirkuit:** Log menunjukkan tahapan: Bootstrapped 0% -> 5% (connecting) -> 10% (connected) -> 14% (handshaking) -> 75% (dirinfo) -> 90% -> 95% -> 100% (done) hanya dalam rentang waktu 3 detik (01:40:02 ke 01:40:05). Respon resmi dari endpoint Tor Project mengonfirmasi {"IsTor":true, "IP":"171.25.193.45"}. Sistem anonimitas beroperasi secara paripurna.

B. Analisis Forensik & Penjelasan Mendalam Arsitektur Anonimitas & OPSEC

Bagian ini merangkum analisis teknis, evaluasi forensik, dan pembedahan mendalam terhadap aspek fundamental arsitektur anonimitas digital, privasi jaringan, mitigasi jejak sistem, serta batasan operasional di lapangan:

Q1: Mengapa pengujian proxychains dengan 'curl ifconfig.me' sering menghasilkan respon '403 Forbidden'? Apakah rute anonimitas mengalami kegagalan?

Jawaban Teknis: Respon HTTP 403 Forbidden BUKAN tanda kegagalan rute, melainkan bukti validitas dan keberhasilan tunneling Tor. Layanan CDN (seperti Cloudflare, Akamai) dan situs publik ifconfig.me memelihara daftar hitam dinamis (Tor Exit Node Blacklist). Ketika request masuk melalui Exit Node Tor (misalnya di Frankfurt atau Amsterdam), Web Application Firewall (WAF) target mendeteksi IP tersebut sebagai simpul publik Tor dan secara otomatis memblokir akses HTTP polos demi mencegah botnet/scraping. Hal ini membuktikan bahwa IP asli operator (ISP lokal) tersembunyi sempurna dan koneksi berhasil dibelokkan keluar yurisdiksi lokal. Pengujian status Tor yang akurat dilakukan melalui endpoint resmi Tor Project (https://check.torproject.org/api/ip) atau layanan yang menerima traffic Tor (icanhazip.com).

Q2: Mengapa peretas profesional tidak membuang waktu konfigurasi manual berjam-jam di lapangan demi mengejar Golden Hours? Strategi arsitektur apa yang digunakan?

Jawaban Teknis: Dalam fase operasional siber, kehilangan Golden Hours (jendela waktu kritis sebelum celah ditambal atau blue team mendeteksi anomali) berakibat fatal terhadap keberhasilan misi. Praktisi dan aktor siber tingkat lanjut tidak pernah mengonfigurasi anonimitas secara manual saat target sudah di depan mata. Mereka mengandalkan tiga metodologi arsitektural: (1) Pre-Staged Snapshot & Ephemeral VMs: Menggunakan mesin virtual siap pakai yang telah dibekukan dalam status snapshot bersih, sehingga lingkungan anonim langsung aktif dalam 5 detik dengan satu klik restore. (2) Infrastructure as Code (IaC) & Deterministic Scripting: Menerapkan skrip otomasi master yang mengeksekusi pengacakan MAC, iptables killswitch, proxy chains, dan DNS hardening dalam satu eksekusi terpadu. (3) Decoupled Attack Infrastructure: Memisahkan identitas mesin lokal dari simpul serangan dengan menyewa jaringan Residential/Rotating Proxy Pools anonim berbayar mata uang kripto privat (Monero), di mana rotasi jutaan IP rumahan terjadi otomatis setiap milidetik.

Q3: Mengapa Tails OS dapat langsung beroperasi secara anonim saat boot pertama kali tanpa konfigurasi manual, dan bagaimana integritas log masa lalunya?

Jawaban Teknis: Tails OS dirancang dengan filosofi 'Amnesic Incognito Live System'. Tails menggunakan citra sistem berkas terkompresi SquashFS yang bersifat read-only (hanya-baca) dan ditandatangani secara kriptografis oleh pengembang. Saat boot pertama kali: (1) Kernel Linux langsung mengeksekusi daemon MAC spoofing otomatis sebelum antarmuka jaringan meminta alamat DHCP ke router lokal. (2) Seluruh outbound network stack dikunci oleh aturan iptables internal (Tails firewall) sehingga tidak ada satu byte pun paket yang dapat keluar kecuali melalui soket Tor bawaan. Mengenai integritas log masa lalu: Tails tidak pernah menulis data ke disk penyimpanan persisten. Seluruh riwayat terminal, cache, file log (/var/log), dan file temporer dialokasikan murni ke memori sementara (tmpfs) di RAM. Begitu sistem dimatikan atau media USB dicabut secara darurat (emergency pull), suplai daya ke transistor RAM terputus dan seluruh catatan forensik musnah seketika (zero trace).

Q4: Bagaimana mekanisme Origin Contamination pada sistem operasi terinstal (seperti Kali Linux VM), dan bagaimana jejak awal instalasi dapat mengekspos operator?

Jawaban Teknis: Setiap sistem operasi yang diinstal ke media penyimpanan permanen (termasuk virtual disk .vdi atau .qcow2) mengalami fenomena Origin Contamination. Saat instalasi awal dan pembaruan paket (apt update/upgrade), sistem telah berkomunikasi langsung dengan repositori mirror menggunakan IP publik asli ISP tanpa lapisan enkripsi anonimitas. Catatan ini tersimpan permanen di: (1) Log sistem (/var/log/syslog, /var/log/auth.log, /var/log/dpkg.log). (2) Pengidentifikasi perangkat keras unik seperti /etc/machine-id dan UUID partisi disk. (3) Catatan DHCP leases (/var/lib/dhcp/) yang mendokumentasikan IP lokal router rumah dan MAC address fisik pertama. Jika disk VM disita oleh penyelidik forensik, artefak ini dapat mengaitkan mesin tersebut ke jaringan fisik operator di masa lalu meskipun sesi serangan terbaru menggunakan Tor. Mitigasi profesional adalah menggunakan Kali Linux Live Forensic Mode (berjalan di RAM tanpa swap disk) atau melakukan pembersihan disk permanen (shred/srm) dan regenerasi machine-id secara berkala.

Q5: Berapa persentase realitas anonimitas sistem ini terhadap server target versus lembaga penegak hukum, dan bagaimana korelasi fisik terjadi?

Jawaban Teknis: Terhadap server target (firewall, web server, IPS korporat), arsitektur ini memberikan tingkat anonimitas 95%-98%—target hanya melihat IP Tor Exit Node di luar negeri dan identitas perangkat acak. Namun terhadap lembaga penegak hukum berkekuatan hukum lintas yurisdiksi (seperti Bareskrim Cyber atau FBI), tingkat perlindungan berada pada kisaran 65%-75%. Penegak hukum tidak memecahkan enkripsi Tor secara matematis, melainkan memanfaatkan Physical Anchors dan Traffic Correlation: (1) Traffic Correlation Attack: Menganalisis ukuran paket dan stempel waktu (timestamp) keluar dari ISP rumah operator lalu mencocokkannya dengan pola paket yang masuk ke Guard Node Tor dan tiba di target. (2) Korelasi Fisik & Telemetri: Keberadaan smartphone aktif di saku atau meja operator yang memancarkan sinyal ke BTS seluler (Cell Tower Triangulation), sinyal Wi-Fi probe request, serta akun personal yang pernah diakses dari jaringan rumah yang sama. Penangkapan aktor siber mayoritas bukan karena enkripsi yang jebol, melainkan kelalaian operasional (OPSEC breach) di dunia nyata.

Q6: Mengapa merotasi IP publik dengan Tor atau VPN menjadi sia-sia jika browser tetap rentan terhadap Canvas Fingerprinting dan WebRTC Leak?

Jawaban Teknis: Mengganti IP publik hanya mengubah alamat pengirim paket pada Layer 3 (Network), namun tidak menyamarkan identitas unik browser pada Layer 7 (Application): (1) WebRTC Leak: Protokol WebRTC (Web Real-Time Communication) menggunakan permintaan STUN melalui protokol UDP. Karena SOCKS proxy standar hanya menangani lalu lintas TCP, paket STUN UDP memotong rute proxy dan mengirimkan IP publik asli ISP serta IP subnet lokal (192.168.x.x) langsung ke server target. (2)

Canvas Fingerprinting: Skrip JavaScript target memerintahkan elemen HTML5 <canvas> untuk merender teks dan bentuk 3D tersembunyi di balik layar. Karena variasi mikro-arsitektur GPU, driver grafis, antialiasing font, dan sistem operasi, hasil render menghasilkan piksel unik dengan nilai hash SHA-256 yang khas. Meskipun operator merotasi IP seribu kali, jika canvas hash-nya identik, server target seketika mengenali bahwa seluruh koneksi tersebut berasal dari mesin fisik yang persis sama. Solusinya adalah mematikan WebRTC (media.peerconnection.enabled = false) dan menggunakan ekstensi Canvas Poisoning yang menyuntikkan noise acak pada piksel setiap sesi.

Q7: Bagaimana jika arsitektur anonimitas dilipatgandakan secara ekstrem (misalnya menjalankan Tor di dalam Tor / Tor-over-Tor)?

Jawaban Teknis: Menjalankan Tor di dalam Tor (Tor-over-Tor) adalah kesalahan arsitektur fatal yang secara tegas dilarang oleh Tor Project. Dampak teknisnya meliputi: (1) TCP-over-TCP Meltdown: Tor berjalan di atas protokol transport TCP yang memiliki mekanisme flow control dan retransmisi paket sendiri. Ketika sirkuit Tor bagian dalam mengalami keterlambatan paket, ia akan memicu retransmisi; sirkuit Tor bagian luar juga mendeteksi hal yang sama dan ikut mengirim ulang paket. Hal ini memicu penumpukan antrian paket secara eksponensial (bufferbloat), latensi membengkak hingga puluhan ribu milidetik (timeout), dan koneksi macet total. (2) Autonomous System (AS) Correlation: Tidak ada jaminan sirkuit luar dan dalam melewati infrastruktur fisik yang terpisah. Jika Guard Node dari sirkuit luar dan Exit Node dari sirkuit dalam berada di bawah kendali penyedia transit/AS yang sama, analisis korelasi lalu lintas justru menjadi jauh lebih mudah bagi analis intelijen. Arsitektur berlapis yang benar bukanlah menduplikasi Tor, melainkan kombinasi heterogen: pemisahan mesin terisolasi (Whonix) dengan jembatan anti-DPI (Obfs4/Snowflake).

Q8: Mengapa arsitektur isolasi dua tingkat (Two-Tier Isolation seperti Whonix Gateway & Workstation) lebih unggul dibandingkan menjalankan seluruh tools anonimitas dalam satu VM biasa?

Jawaban Teknis: Ketika aplikasi peramban, tools eksploitasi, dan modul anonimitas berjalan pada sistem operasi yang sama (single-tier environment), terdapat risiko kritis Kernel Exploit & Direct Socket Leak: Jika target mengeksploitasi celah zero-day pada browser dan mendapatkan akses root lokal, kode jahat tersebut dapat mematikan iptables, menghentikan layanan Tor, dan membuka soket RAW langsung ke antarmuka kartu jaringan untuk membocorkan IP publik asli operator secara diam-diam. Sebaliknya, pada Arsitektur Dua Tingkat (Two-Tier Architecture seperti Whonix): (1) Whonix-Workstation (tempat operator beraktivitas) secara fisik/virtual tidak memiliki kartu jaringan eksternal dan tidak mengetahui IP publik asli, hanya terhubung melalui jaringan internal terisolasi ke Whonix-Gateway. (2) Whonix-Gateway bertindak sebagai router Tor murni yang memproses seluruh lalu lintas keluar. Bahkan jika Workstation diretas hingga level root sekalipun, mesin tersebut secara arsitektural mustahil membocorkan IP asli karena tidak memiliki antarmuka ke internet publik.

C. Skrip Otomatisasi Lengkap 1-Ketik (One-Click Setup Script)

Untuk menghindari proses konfigurasi manual yang memakan waktu, berikut adalah skrip otomasi master yang dirancang berurutan dari layer terendah (Layer 2 MAC) hingga layer tertinggi (Layer 8 Timezone/RAM):

```
$ cat << 'EOF' > /usr/local/bin/setup_stealth_master.sh
#!/bin/bash
set -e

echo "[*] Memulai Pembangunan Lingkungan Anonim Terintegrasi..."

# 1. LAYER 2 (DATA LINK): PENGACAKAN IDENTITAS FISIK
echo "[1/6] Layer 2: Mengacak MAC Address kartu jaringan eth0..."
ip link set eth0 down
macchanger -r eth0
ip link set eth0 up

# 2. LAYER 3 & 4 (NETWORK & TRANSPORT): MENYIAPKAN DEPENDENSI TOR & SOCKS
echo "[2/6] Layer 3-4: Memastikan Master Service Tor Aktif..."
apt-get update -y > /dev/null 2>&1
apt-get install -y secure-delete tor macchanger mat2 > /dev/null 2>&1
systemctl restart tor

# 3. KERNEL LAYER (PACKET FILTERING): TRANSPARENT PROXY & IPTABLES
echo "[3/6] Kernel Layer: Mengaktifkan Anonsurf Transparent Proxying..."
anonsurf start > /dev/null 2>&1

# 4. LAYER 7 (APPLICATION): BROWSER HARDENING ANTI-FINGERPRINT & WEBRTC
echo "[4/6] Layer 7: Mengunci Firefox Anti-WebRTC & Canvas Poisoning..."
FF_PROFILE_DIR=$(find /root/.mozilla/firefox -name "*.default-esr" -o -name "*.default" | head -n 1)
if [ -n "$FF_PROFILE_DIR" ]; then
    cat << 'PREF' >> "$FF_PROFILE_DIR/user.js"
user_pref("media.peerconnection.enabled", false);
user_pref("privacy.resistFingerprinting", true);
user_pref("privacy.trackingprotection.enabled", true);
PREF
    echo "    -> Setelan user.js Firefox berhasil disuntikkan."
fi

# 5. LAYER 8 (SYSTEM & BEHAVIORAL): TIMEZONE UTC & LOCALE SANITIZATION
echo "[5/6] Layer 8: Mengunci Zona Waktu Sistem ke UTC..."
timedatectl set-timezone UTC

# 6. VERIFIKASI IDENTITAS AKHIR
echo "[6/6] Selesai! Verifikasi Koneksi Anonim Paripurna:"
anonsurf myip

EOF
chmod +x /usr/local/bin/setup_stealth_master.sh
```

Penjelasan Arti Setiap Baris Perintah Skrip:

1. `ip link set eth0 down && macchanger -r eth0 && ip link set eth0 up`: Mematikan interface jaringan untuk mengganti 12 digit alamat MAC fisik ke alamat acak sebelum ada paket yang terkirim ke router lokal.

2. `apt-get install -y secure-delete tor macchanger mat2`: Memasang seluruh perkakas inti dalam satu kali panggilan tanpa memerlukan interaksi pengguna.

3. `systemctl restart tor`: Memulai daemon Tor untuk membuka port lokal SOCKS5 9050, TransPort 9040, dan DNSPort 53.

4. `anonsurf start`: Mengaktifkan aturan firewall iptables yang mencegat semua paket TCP dan DNS keluar, membelokkannya ke Tor, serta memblokir paket non-TCP (UDP/ICMP leak).

5. `user_pref("media.peerconnection.enabled", false)`: Mematikan mesin WebRTC di Firefox agar situs target tidak bisa melakukan STUN query untuk membongkar IP lokal/publik asli.

6. `user_pref("privacy.resistFingerprinting", true)`: Mengaktifkan canvas poisoning, membulatkan resolusi layar, dan mengunci identitas browser ke versi umum.

7. `timedatectl set-timezone UTC`: Mengubah jam internal sistem ke Greenwich Time (UTC) agar analis tidak bisa menebak zona waktu geografis operator (WIB/Indonesia).

D. Cheatsheet Komprehensif Perintah Operasional Cepat

Melihat MAC address fisik	`ip link show eth0`	Cek baris link/ether OUI
Mengacak MAC address	`ip link set eth0 down && macchanger -r eth0 && ip link set eth0 up`	Reset otomatis saat mesin reboot
Menjalankan master service Tor	`systemctl start tor`	Membuka SOCKS5 di 127.0.0.1:9050
Mengecek status service Tor	`systemctl status tor`	Pastikan status active (running)
Cek IP publik asli (Benchmark)	`curl ifconfig.me`	Koneksi terbuka tanpa lapisan proxy
Cek IP via Proxychains	`proxychains curl https://icanhazip.com`	Rute aplikasi individual via SOCKS5
Verifikasi sirkuit Tor	`proxychains curl https://check.torproject.org/api/ip`	Respon sukses: {"IsTor":true}
Port scanning anonim	`proxychains nmap -sT -Pn -p <ports> <target>`	Wajib pakai flag -sT (TCP connect)
Mulai transparent proxy global	`anonsurf start`	Mengalihkan semua TCP & DNS via iptables
Cek IP aktif seluruh sistem	`anonsurf myip`	Tanpa perlu mengetikkan 'proxychains'
Ganti identitas/IP on-the-fly	`anonsurf change`	Mengirim sinyal NEWNYM ke port kontrol
Cek status transparent proxy	`anonsurf status`	Melihat status relai dan tabel iptables
Hentikan sesi & sanitasi RAM	`anonsurf stop`	Flush iptables, reset DNS, dan wipe cache
Matikan WebRTC di Firefox	`about:config ->`	Mencegah kebocoran IP

	`media.peerconnection.enabled = false`	lokal/publik via STUN
Acak Canvas Fingerprint	`about:config -> privacy.resistFingerprinting = true`	Poisoning canvas hash, lock timezone UTC
Inspeksi metadata file	`mat2 -s <nama_file>`	Membongkar tag EXIF, Author, dan GPS
Sanitasi/Hapus metadata	`mat2 <nama_file>`	Menghasilkan file aman berakhiran .cleaned
Instalasi transport bridges	`apt install -y obfs4proxy snowflake-client`	Mempersiapkan binary obfuscator Tor
Reset konfigurasi bridge torrc	`sed -i '/TOR PLUGGABLE TRANSPORTS/,$d' /etc/tor/torrc`	Memulihkan torrc ke mode stabil cepat
Inspeksi siklus bootstrap Tor	`journalctl -u tor@default -n 15 --no-pager`	Memverifikasi Bootstrapped 100% (done)
Deep clean jejak log & history	`cat /dev/null > ~/.bash_history && rm -rf /var/log/*`	Menghapus jejak artefak masa lalu